FACULTEIT INGENIEURSWETENSCHAPPEN
EN ARCHITECTUUR

# Een schaalbare multi-agentarchitectuur voor Enterprise Search

Arne Albrecht
Studentennummer: 02105980

Promotoren: prof. dr. ir. Filip De Turck, dhr. Joris Steppe (Easi)
Begeleider: dhr. Dorian Féaux (Easi)

Masterproef voorgelegd voor het behalen van de graad Master of Science in de industriële wetenschappen: informatica

Academiejaar 2025-2026

UNIVERSITEIT
GENT

# Abstract

**Titel:** Een Schaalbare Multi-Agentarchitectuur voor Enterprise Search

**Student:** Arne Albrecht

**Opleiding:** Master in de industriële wetenschappen: informatica

**Bedrijfspartner:** Easi

**Promotor(en) en begeleider(s):** Prof. dr. ir. Filip De Turck, dhr. Joris Steppe (Easi), dhr. Dorian Féaux (Easi)

Binnen moderne organisaties bevindt relevante informatie zich verspreid over verschillende systemen, zoals e-mailomgevingen, documentplatformen en CRM-toepassingen. Daardoor wordt het steeds moeilijker om snel een volledig en bruikbaar antwoord op een informatievraag te formuleren. Deze masterproef onderzoekt hoe een multi-agentarchitectuur kan worden ingezet om bedrijfsdata uit meerdere systemen op een veilige en efficiënte manier te doorzoeken. Hiervoor werd een prototype ontwikkeld met een centrale orchestrator en drie gespecialiseerde subagents voor Microsoft Graph, Salesforce en SmartSales. De communicatie met de onderliggende databronnen verloopt via het Model Context Protocol (MCP), terwijl Microsoft Agent Framework en Azure OpenAI gebruikt werden voor de agentlogica. Het ontwikkelde prototype toont aan dat deze architectuur technisch haalbaar is en dat informatie uit meerdere systemen op een gecontroleerde en uitbreidbare manier kan worden samengebracht. De evaluatie gebeurde aan de hand van testqueries waarbij onder meer correctheid van antwoorden, query routing, responstijd en tokenverbruik werden onderzocht. De eerste resultaten geven aan dat de voorgestelde aanpak potentieel heeft voor enterprise search binnen bedrijfsomgevingen, maar tonen ook aan dat de uiteindelijke kwaliteit sterk afhangt van de betrouwbaarheid en maturiteit van de onderliggende agent- en toolimplementaties. Deze masterproef levert daarmee een proof-of-concept en een basis voor verdere verfijning en uitbreiding in een volgende ontwikkelfase.

**Trefwoorden:** multi-agentarchitectuur, enterprise search, Agentic AI, Model Context Protocol, Large Language Model

# Contents

**Abstract** ii

**List of Figures** vi

**List of Tables** vii

**1 Inleiding** 1
1.1 Context en motivatie . . . 1
1.2 Probleemstelling . . . 2
1.3 Onderzoeksvragen . . . 2
1.4 Doelstelling van de scriptie . . . 3
1.5 Opbouw van de scriptie . . . 3

**2 Achtergrond en literatuurstudie** 4
2.1 Enterprise search binnen organisaties . . . 4
2.2 Large Language Models en hun beperkingen . . . 4
2.3 AI-agents en multi-agent systemen . . . 5
2.4 Agent frameworks . . . 6
2.5 Model Context Protocol . . . 7
2.6 Nood aan governance bij toegang tot bedrijfsdata . . . 8

**3 Probleemanalyse en vereisten** 9
3.1 Verspreide data over meerdere systemen . . . 9
3.2 Uitdagingen rond integratie, authenticatie en datamodellen . . . 9
3.3 Beperkingen van klassieke zoekmethoden . . . 10
3.4 Risico's van LLM-gebaseerde systemen . . . 11
3.5 Functionele en niet-functionele vereisten . . . 11
3.5.1 Functionele vereisten . . . 11
3.5.2 Niet-functionele vereisten . . . 12

**4 Architectuurontwerp** 13

4.1 Overzicht van de voorgestelde architectuur . . . 13
4.2 Beschrijving van de componenten . . . 14
4.2.1 Orchestrator . . . 14
4.2.2 Gespecialiseerde agents . . . 15
4.2.3 MCP-server of toollaag per databron . . . 15
4.2.4 Onderliggende databronnen . . . 15
4.3 Communicatie- en queryflow . . . 16
4.4 Veiligheids- en toegangsmodel . . . 17
4.5 Verantwoording van de architecturale keuzes . . . 17

**5 Implementatie 18**
5.1 Gebruikte technologieën en stack . . . 18
5.2 Implementatie van de orchestrator . . . 19
5.3 Implementatie van de Microsoft Graph-Agent . . . 19
5.4 Implementatie van de Salesforce-Agent . . . 20
5.5 Implementatie van de SmartSales-Agent . . . 21
5.6 Integratie tussen agents en databronnen . . . 21
5.7 Beperkingen van de huidige implementatie . . . 22

**6 Evaluatiemethodologie 23**
6.1 Evaluatie-opzet . . . 23
6.2 Testscenario's en querycategorieën . . . 24
6.2.1 Samenstelling van de testset . . . 24
6.2.2 Overzicht van de querycategorieën . . . 24
6.3 Evaluatie van correctheid . . . 25
6.4 Evaluatie van query routing . . . 25
6.5 Evaluatie van performantie en kost . . . 26
6.5.1 Evaluatie van de responstijd . . . 26
6.5.2 Evaluatie van het tokenverbruik . . . 26
6.6 Beperkingen van de evaluatie . . . 26

**7 Evaluatieresultaten 29**
7.1 Algemene resultaten . . . 29
7.2 Evaluatie van correctheid . . . 30
7.3 Evaluatie van query routing . . . 32
7.4 Evaluatie van performantie en kost . . . 33
7.4.1 Evaluatie van de responstijd . . . 33
7.4.2 Evaluatie van het tokenverbruik . . . 34

7.5 Beperkingen van de evaluatie . . . . . 34

**8 Werkpunten voor een volgende versie** **36**

8.1 Algemene uitbreidingen . . . . . 36

8.2 Uitbreidingen voor de Microsoft Graph-Agent . . . . . 36

8.3 Uitbreidingen voor de Salesforce-Agent . . . . . 37

8.4 Uitbreidingen voor de SmartSales-Agent . . . . . 37

8.5 Uitbreidingen voor de orchestrator . . . . . 37

8.6 Uitgebreidere evaluatie . . . . . 38

**References** **39**

**Appendices** **41**

Bijlage A – Volledige lijst van testqueries . . . . . 42

# List of Figures

4.1 Overzicht van de voorgestelde multi-agentarchitectuur: de orchestrator coördineert drie gespecialiseerde subagents (Graph Agent, Salesforce-Agent en SmartSales-Agent), die elk via een afzonderlijke MCP-server communiceren met hun onderliggende databron. . . . . 14
4.2 Schematische voorstelling van de communicatie- en queryflow: de gebruikersvraag wordt ontvangen door de orchestrator, die de relevante agents aanspreekt via de bijhorende MCP-servers. De deelresultaten worden samengevoegd tot één eindantwoord voor de gebruiker. . . . . 16

# List of Tables

6.1 Overzicht van de testqueries per agent en subcategorie . . . . . 28

7.1 Overzicht van evaluatieresultaten per agent (meest recente run, 31 maart 2026) . . . . . 29
7.2 Routingresultaten per categorie (OrchestratorAgent, run 0f9f37a4) . . . . . 33

# 1

# Inleiding

## 1.1 Context en motivatie

De digitalisering van bedrijfsprocessen heeft ertoe geleid dat organisaties steeds grotere hoeveelheden data genereren en opslaan. Deze informatie bevindt zich vaak verspreid over verschillende systemen en applicaties, zoals documentbeheersystemen, Customer Relationship Management (CRM)-platformen, e-mailomgevingen en andere interne bedrijfsapplicaties. Hoewel deze systemen elk waardevolle informatie bevatten, zijn ze doorgaans ontwikkeld als afzonderlijke oplossingen met eigen databronnen, interfaces en toegangsmechanismen. Hierdoor wordt het voor gebruikers steeds moeilijker om snel en efficiënt relevante informatie terug te vinden wanneer deze zich over meerdere systemen verspreid bevindt.

Binnen veel organisaties is informatieopzoeking daarom nog steeds een tijdrovend proces. Gebruikers moeten vaak verschillende applicaties afzonderlijk doorzoeken om een volledig beeld te krijgen van een bepaalde klant, document of communicatie. Naarmate het aantal gebruikte softwareplatformen en de hoeveelheid opgeslagen data blijven toenemen, wordt dit probleem steeds groter. Het gebrek aan een uniforme manier om informatie uit verschillende systemen te combineren vormt dan ook een belangrijke uitdaging binnen moderne Information Technology (IT)-omgevingen.

De recente opkomst van generatieve Artificial Intelligence (AI) en Large Language Models (LLMs) biedt nieuwe mogelijkheden om met deze problematiek om te gaan. Dergelijke modellen zijn in staat om natuurlijke taal te begrijpen en complexe vragen van gebruikers te interpreteren. Daardoor wordt het mogelijk om zoekinterfaces te bouwen waarin gebruikers vragen in natuurlijke taal kunnen stellen, waarna het systeem relevante informatie ophaalt en samenbrengt in een bruikbaar antwoord.

In deze scriptie wordt zo'n zoekoplossing benaderd als een *global search agent*: een AI-gestuurde zoeklaag waarmee een gebruiker via natuurlijke taal informatie kan opvragen over meerdere onderliggende bedrijfssys-

temen heen, zonder elk systeem afzonderlijk te moeten doorzoeken.

## 1.2 Probleemstelling

Hoewel generatieve AI veel potentieel heeft, is de integratie ervan in bedrijfsomgevingen niet vanzelfsprekend. Taalmodellen hebben standaard geen directe toegang tot interne databronnen en beschikken niet over ingebouwde mechanismen om met verschillende bedrijfsapplicaties te communiceren. Bovendien moeten organisaties rekening houden met belangrijke aspecten zoals beveiliging, toegangscontrole, privacy en governance van bedrijfsdata.

Daarnaast brengen bedrijfsomgevingen bijkomende technische uitdagingen met zich mee. Data bevindt zich verspreid over meerdere systemen met verschillende Application Programming Interfaces (APIs), authenticatiemechanismen en datamodellen. Daardoor volstaat één enkel taalmodel niet om op een betrouwbare manier als zoeklaag bovenop meerdere systemen te functioneren. Het doel is dus om een oplossing te maken die niet alleen veilige toegang biedt, maar ook meerdere databronnen kan integreren en vragen van gebruikers kan routeren naar de juiste systemen.

## 1.3 Onderzoeksvragen

Deze scriptie vertrekt vanuit de centrale onderzoeksvraag:

> *Hoe kan een multi-agentarchitectuur gebruikt worden om bedrijfsdata uit meerdere systemen op een veilige en efficiënte manier te doorzoeken?*

Vanuit deze centrale vraag worden de volgende deelvragen onderzocht:

- Hoe kunnen AI-agents veilig toegang krijgen tot bedrijfsdata in verschillende systemen?
- Hoe kunnen meerdere databronnen met uiteenlopende APIs en datamodellen geïntegreerd worden binnen één architectuur?
- Welke multi-agentarchitectuur is het meest geschikt voor een global search agent binnen een bedrijfsomgeving?
- In welke mate kan een multi-agent systeem vragen van gebruikers correct interpreteren, routeren en beantwoorden over verschillende databronnen heen?

## 1.4 Doelstelling van de scriptie

Het doel van deze masterproef is het ontwerpen, implementeren en evalueren van een multi-agentarchitectuur voor een global search agent binnen bedrijfsomgevingen. Concreet wordt een prototype ontwikkeld dat als zoeklaag dient bovenop verschillende applicaties, waarbij AI-agents gebruikt worden om de informatie uit meerdere systemen op te halen, waarna de resultaten worden gecombineerd tot een samenhangend antwoord voor de gebruiker.

Deze masterproef wordt uitgevoerd in samenwerking met het bedrijf Easi en maakt gebruik van moderne AI- en cloudtechnologieën, waaronder Azure AI Foundry en het Model Context Protocol (MCP). Het ontwikkelde prototype dient als proof-of-concept voor een schaalbare architectuur waarmee organisaties generatieve AI op een gecontroleerde en uitbreidbare manier kunnen integreren binnen hun bestaande IT-landschap.

## 1.5 Opbouw van de scriptie

De rest van deze scriptie is als volgt opgebouwd. Hoofdstuk 2 bespreekt de relevante achtergrond en literatuur rond enterprise search, LLMs, AI-agents, multi-agent systemen en MCP. Hoofdstuk 3 analyseert het probleem van informatiefragmentatie binnen bedrijfsomgevingen en definieert de functionele en niet-functionele vereisten voor een global search agent. Hoofdstuk 4 beschrijft het ontwerp van de voorgestelde architectuur en verantwoordt de gemaakte architecturale keuzes. Hoofdstuk 5 behandelt de implementatie van het prototype en de integratie met de verschillende databronnen. In hoofdstuk 6 wordt het systeem geëvalueerd aan de hand van meerdere testscenario's en querycategorieën.

# 2

# Achtergrond en literatuurstudie

## 2.1 Enterprise search binnen organisaties

Met de term enterprise search wordt verwezen naar zoekoplossingen binnen organisaties waarmee gebruikers informatie proberen terug te vinden, zoals documenten, e-mails, klanten, tickets, dossiers en andere bedrijfsgegevens. In moderne bedrijfsomgevingen bevindt deze informatie zich doorgaans verspreid over verschillende systemen en applicaties, zoals documentplatformen, CRM-systemen, Enterprise Resource Planning (ERP)-systemen, interne databanken en collaboration tools.

Deze spreiding over meerdere systemen bemoeilijkt het zoeken aanzienlijk. De betrokken systemen hanteren vaak eigen databronnen, interfaces, metadata, APIs en authenticatiemethoden. Daardoor is informatie niet alleen verdeeld over verschillende platformen, maar ook op uiteenlopende manieren gestructureerd. In de praktijk betekent dit dat gebruikers vaak meerdere systemen afzonderlijk moeten doorzoeken om een volledig beeld te krijgen van een klant, project, document of communicatie.

Bestaande enterprise search-oplossingen zijn in veel gevallen nog sterk gebaseerd op trefwoorden en leveren vooral losse resultatenlijsten op. Zulke systemen slagen er niet altijd in om de context van een gebruikersvraag goed te begrijpen of informatie uit meerdere bronnen op een zinvolle manier te combineren. Vooral bij complexere zoekvragen stoten klassieke zoekmethoden daardoor op hun limieten. Dit creëert de nood aan intelligentere zoekarchitecturen die niet alleen informatie kunnen terugvinden, maar ook de context van een vraag beter kunnen interpreteren.

## 2.2 Large Language Models en hun beperkingen

LLMs bieden nieuwe mogelijkheden voor moderne zoeksystemen. In tegenstelling tot klassieke zoekmethoden, die vaak sterk afhankelijk zijn van trefwoorden, kunnen LLMs natuurlijke taal begrijpen en genereren [1]. Daardoor kunnen gebruikers hun vraag in natuurlijke taal formuleren, terwijl het model de intentie achter die vraag

interpreteert. Bovendien kunnen LLMs informatie samenvatten, verschillende stukken context combineren en een bruikbaar antwoord formuleren in plaats van enkel een lijst van zoekresultaten terug te geven.

Binnen enterprise search maakt dit het gebruik van LLMs bijzonder relevant. Ze kunnen niet alleen helpen bij het beter interpreteren van gebruikersvragen, maar ook bij het structureren en samenbrengen van informatie uit verschillende bronnen. Daardoor ontstaat de mogelijkheid om te evolueren van klassieke resultatenlijsten naar zoekinterfaces die gebruikers directer ondersteunen bij het terugvinden van relevante informatie.

Tegelijk volstaat een LLM op zich niet als oplossing voor enterprise search binnen bedrijfsomgevingen. Een model heeft standaard geen ingebouwde toegang tot private bedrijfsdata en kan zonder bijkomende integratie geen informatie ophalen uit interne systemen zoals e-mailomgevingen, CRM-platformen of documentbeheersystemen. Daarnaast is de kennis van een LLM beperkt tot de trainingsdata en ontbreekt zonder externe context toegang tot actuele of domeinspecifieke informatie. Ook zijn de gegenereerde antwoorden probabilistisch van aard, wat kan leiden tot inconsistente output of hallucinaties. Ten slotte beschikken LLMs niet over ingebouwde mechanismen voor API-integratie, authenticatie, autorisatie en het respecteren van toegangsrechten. Daarom is een bijkomende architecturale laag nodig die relevante context ophaalt, toegang tot bedrijfsdata regelt en deze op een gecontroleerde manier aan het model beschikbaar stelt.

## 2.3 AI-agents en multi-agent systemen

Een AI-agent kan worden omschreven als een softwarecomponent die autonoom of semi-autonoom taken uitvoert namens een gebruiker of applicatie [2]. In tegenstelling tot een LLM, dat op zichzelf hoofdzakelijk input verwerkt en tekstuele output genereert, kan een agent ook acties ondernemen. Daarbij kan een agent gebruikmaken van mogelijkheden zoals toolgebruik, geheugen, planning en contextbeheer [3].

Het gedrag van een agent verloopt typisch via een iteratief patroon van redeneren en handelen, ook bekend als het ReAct-patroon (*Reason + Act*) [4]. Daarin redeneert de agent intern over de huidige context, voert een gerichte actie uit via tools of externe systemen, en verwerkt de terugkoppeling om zijn interne toestand bij te werken. Dit cyclische patroon stelt agents in staat om niet uitsluitend op hun interne kennis te steunen, maar ook actief externe informatie op te vragen en meerdere stappen te plannen.

In een bedrijfscontext is dat onderscheid belangrijk. Waar een LLM op zichzelf geen rechtstreekse interactie heeft met externe systemen, kan een agent databronnen aanspreken, informatie ophalen, tussenresultaten bijhouden en deze verwerken in verdere stappen. De LLM fungeert daarbij als redeneer- of beslissingscomponent, terwijl de agent als geheel de bijkomende taaklogica en systeeminteractie voorziet. Hierdoor zijn agents bijzonder geschikt voor gebruik in omgevingen waar informatie verspreid zit over meerdere databronnen en waar complexe workflows verschillende systemen met elkaar moeten verbinden.

Een multi-agent systeem is een architectuur waarbij meerdere gespecialiseerde agents samenwerken om een

complexere taak uit te voeren [5, 2]. In plaats van één enkele agent die alle verantwoordelijkheden op zich neemt, wordt de functionaliteit verdeeld over verschillende agents met elk een eigen rol of specialisatie. In de context van enterprise search kan bijvoorbeeld voor elke databron of elk extern systeem een aparte agent worden voorzien, met eigen integratielogica, authenticatiestromen, Representational State Transfer (REST)-clients en datamodellen.

Deze opsplitsing biedt verschillende voordelen. In de eerste plaats zorgt dit voor een duidelijke scheiding van verantwoordelijkheden, waardoor de architectuur modulairder en beter onderhoudbaar wordt. Elke agent kan zich toespitsen op één specifieke databron of taak, wat het eenvoudiger maakt om nieuwe systemen toe te voegen. Daarnaast laat een multi-agent aanpak toe om query routing explicieter te organiseren, waarbij een centrale orchestrator bepaalt welke agents moeten worden aangesproken om een bepaalde gebruikersvraag te beantwoorden [6].

Tegelijk brengt een multi-agentarchitectuur ook belangrijke uitdagingen met zich mee. Doordat het systeem uit meerdere samenwerkende componenten bestaat, neemt de complexiteit van de architectuur toe. Dit maakt debugging en foutanalyse moeilijker dan in een meer monolithische architectuur. Bovendien kunnen tokenkost en latency snel oplopen wanneer voor één gebruikersvraag meerdere LLM-aanroepen nodig zijn. Tot slot moet ook de beveiliging over meerdere componenten heen bewaakt worden, wat bijkomende eisen stelt aan authenticatie, autorisatie en controle op gegevensuitwisseling tussen agents.

## 2.4 Agent frameworks

Een agent framework is een softwarelaag die helpt bij het bouwen, uitvoeren en coördineren van agents. Dergelijke frameworks voorzien mechanismen voor onder meer toolgebruik, state- en contextbeheer, samenwerking tussen meerdere agents, workflow-uitvoering, observability en integratie met externe systemen. Binnen deze masterproef werden drie frameworks nader bekeken: LangGraph, Semantic Kernel en Microsoft Agent Framework.

Bij de keuze van een geschikt framework zijn in deze context verschillende criteria relevant: de mate van controle over de uitvoeringsflow, de flexibiliteit van het gedrag van de agent, de ondersteuning van multi-agent samenwerking en de fit voor gebruik in een enterpriseomgeving. Aangezien de voorgestelde architectuur meerdere gespecialiseerde agents combineert met externe databronnen, volstaat het niet dat een framework enkel flexibel is. Het moet ook toelaten om de interactie tussen agents, het gebruik van tools en de uitvoeringsflow op een duidelijke manier te organiseren.

LangGraph is in de eerste plaats sterk in scenario's waarin de logica expliciet gemodelleerd moet worden als een graaf van stappen en beslissingspunten [7]. Het framework laat ontwikkelaars toe om nodes, edges en conditionele overgangen te definiëren, waardoor workflows op een gestructureerde en controleerbare manier kunnen

worden opgebouwd. LangGraph ondersteunt bovendien interrupts, checkpoints en human-in-the-loop interacties, wat het geschikt maakt voor voorspelbare en controleerbare processen [8]. Tegelijk is deze aanpak minder flexibel, omdat de globale flow grotendeels vooraf moet worden vastgelegd.

Semantic Kernel vertrekt vanuit een aanpak waarbij een taalmodel tijdens de uitvoering mee bepaalt welke functies of hulpmiddelen worden gebruikt om een taak op te lossen [9]. Daardoor is het framework vooral interessant in situaties waar flexibiliteit belangrijk is. Een voordeel van deze aanpak is dat het systeem zich gemakkelijker kan aanpassen aan uiteenlopende vragen en taken. Tegelijk heeft dit ook een nadeel: een groter deel van de beslissingslogica ligt bij het taalmodel zelf, wat het moeilijker maakt om het systeem volledig voorspelbaar en reproduceerbaar te houden.

Microsoft Agent Framework sluit gedeeltelijk aan bij dezelfde ideeën, maar legt meer nadruk op structuur en controle binnen agentische systemen [10, 11]. Het framework is ontworpen om samenwerking tussen meerdere agents beter te organiseren en om duidelijker vast te leggen welke rol elke agent heeft binnen het geheel. Hierdoor ontstaat een aanpak die enerzijds voldoende flexibiliteit biedt, maar anderzijds ook meer controle geeft over hoe agents samenwerken en welke acties zij mogen uitvoeren. Dat maakt het framework bijzonder relevant voor toepassingen in een enterprise context, waar betrouwbaarheid, controle en uitbreidbaarheid centraal staan.

Bij de vergelijking van deze frameworks valt op dat elk framework andere accenten legt. LangGraph biedt veel controle en voorspelbaarheid, maar is minder geschikt voor dynamische agentinteracties. Semantic Kernel biedt veel flexibiliteit, maar laat een groter deel van de control flow afhangen van het taalmodel. Microsoft Agent Framework bevindt zich tussen beide benaderingen en combineert expliciete workflowsturing met ondersteuning voor multi-agent samenwerking. Om die reden werd binnen deze masterproef gekozen voor Microsoft Agent Framework als basis voor het prototype.

## 2.5 Model Context Protocol

Het MCP is een gestandaardiseerd protocol waarmee taalmodellen en externe systemen op een uniforme manier met elkaar kunnen communiceren [12]. Via MCP kan een model tools of databronnen aanspreken zonder rechtstreeks gekoppeld te zijn aan een specifiek AI-framework of een bepaalde vendor. Het protocol vormt zo een gemeenschappelijke laag tussen taalmodellen en externe systemen. Een bruikbare vergelijking is die met USB-C in de hardwarewereld: één uniforme aansluiting die het mogelijk maakt uiteenlopende componenten met elkaar te verbinden.

Een belangrijk probleem dat MCP probeert op te lossen, is het zogenaamde MxN-integratieprobleem. Zonder een gemeenschappelijke standaard moet voor elke combinatie van model, framework en databron een aparte koppeling worden gebouwd. Dat leidt snel tot een groot aantal integraties die moeilijk onderhoudbaar en uitbreidbaar zijn. MCP biedt hiervoor een uniforme interface waarop zowel applicaties als externe tools en databronnen kun-

nen worden aangesloten.

De architectuur van MCP bestaat uit drie centrale componenten: een host, een client en een server [13, 14]. De host is de AI-applicatie die de volledige interactie beheert, waaronder gebruikersinput, LLM-aanroepen, contextbeheer en de selectie van geschikte tools. Binnen deze host bevinden zich één of meerdere MCP-clients. Een client onderhoudt telkens een één-op-éénverbinding met een MCP-server en verzorgt de communicatie tussen het model en die server. Een MCP-server is op zijn beurt een extern programma of een service die tools, resources of andere functionaliteiten beschikbaar stelt.

Tegelijk kent ook MCP enkele beperkingen en aandachtspunten. Hoewel het protocol de communicatie met externe systemen standaardiseert, lost het niet automatisch alle uitdagingen rond beveiliging, toegangscontrole en governance op. Er moet nog steeds expliciet worden vastgelegd welke tools toegankelijk zijn, welke acties mogen worden uitgevoerd en welke data aan een model mag worden doorgegeven. MCP biedt dus vooral een gestandaardiseerde integratielaag, maar vormt op zichzelf geen volledige oplossing voor alle architecturale en beveiligingsgerelateerde uitdagingen.

## 2.6 Nood aan governance bij toegang tot bedrijfsdata

Ten slotte speelt ook governance een belangrijke rol. Organisaties moeten niet alleen kunnen bepalen tot welke data een agent toegang heeft, maar ook welke tools of acties binnen het systeem toegelaten zijn en hoe de werking van het systeem gecontroleerd kan worden. In dat verband zijn logging, traceerbaarheid en auditbaarheid belangrijk [15]. Wanneer een systeem antwoorden formuleert op basis van meerdere databronnen, moet achteraf in zekere mate kunnen worden nagegaan welke informatie werd geraadpleegd en via welke componenten die werd verwerkt.

# 3

# Probleemanalyse en vereisten

## 3.1 Verspreide data over meerdere systemen

Binnen organisaties is relevante bedrijfsinformatie zelden gecentraliseerd opgeslagen in één systeem. In de praktijk bevindt informatie zich verspreid over verschillende applicaties, zoals e-mailomgevingen, documentplatformen, CRM-systemen, samenwerkingsomgevingen en andere interne tools. Elk van deze systemen ondersteunt een specifiek bedrijfsproces en beheert daarbij zijn eigen gegevensverzameling. Hierdoor ontstaat een gefragmenteerd informatielandschap waarin kennis en context over dezelfde entiteit, zoals een klant, project of medewerker, over meerdere systemen verdeeld kunnen zijn.

Deze spreiding van data bemoeilijkt het terugvinden van informatie aanzienlijk. Gebruikers weten vaak niet op voorhand in welk systeem de gezochte informatie zich bevindt en moeten daarom meerdere applicaties afzonderlijk raadplegen. In veel gevallen volstaat informatie uit één bron niet om een volledig antwoord te formuleren. Relevante context kan bijvoorbeeld deels aanwezig zijn in e-mails, deels in documenten en deels in CRM-gegevens. Het verzamelen van deze informatie vraagt dus niet alleen zoekwerk binnen afzonderlijke systemen, maar ook een manuele combinatie van resultaten uit verschillende bronnen.

Deze spreiding maakt het zoekproces ook complexer voor de gebruikers. Zij moeten niet alleen meerdere tools gebruiken, maar ook begrijpen hoe informatie in elk systeem gestructureerd is en welke terminologie daar wordt gebruikt. Naarmate organisaties meer digitale platformen gebruiken, neemt deze complexiteit verder toe.

Daardoor ontstaat de nood aan een oplossing die informatie over verschillende systemen heen toegankelijk kan maken, zonder dat gebruikers elk systeem afzonderlijk moeten doorzoeken.

## 3.2 Uitdagingen rond integratie, authenticatie en datamodellen

Het samenbrengen van informatie uit meerdere systemen is niet alleen een organisatorische, maar ook een technische uitdaging. Elk systeem stelt data op een eigen manier beschikbaar, bijvoorbeeld via specifieke REST-APIs,

Software Development Kits (SDKs) of andere integratiemechanismen. Deze interfaces verschillen in structuur, mogelijkheden en beperkingen. Daardoor volstaat één generieke koppeling niet. Voor elke databron moet rekening worden gehouden met de concrete technische eigenschappen van het betrokken platform.

Een eerste moeilijkheid betreft de integratie. Verschillende systemen hanteren eigen endpoints, queryparameters, filtermogelijkheden en antwoordformaten. Sommige systemen ondersteunen uitgebreide zoekfunctionaliteit, terwijl andere slechts beperkte opvragingen toelaten. Ook foutafhandeling, rate limiting en paginering verschillen van platform tot platform. MCP kan die verschillen gedeeltelijk abstraheren via een uniforme interface, maar neemt de systeem-specifieke implementatielogica niet weg.

Daarnaast vormt authenticatie een belangrijke uitdaging. Toegang tot bedrijfsdata verloopt doorgaans via beveiligde authenticatie zoals Open Authorization (OAuth) 2.0, access tokens of andere identityflows. Een geïntegreerd zoeksysteem moet deze mechanismen correct ondersteunen om veilig toegang te bieden tot de onderliggende systemen. Tegelijk moet het systeem rekening houden met de rechten van de ingelogde gebruiker. Indien een gebruiker geen toegang heeft tot bepaalde informatie in het bronsysteem, mag deze informatie niet ineens beschikbaar zijn via het systeem.

Doordat informatie uit verschillende systemen afkomstig is, worden gelijkaardige entiteiten in elke applicatie vaak anders gemodelleerd. Ze kunnen verschillen in benaming, structuur, beschikbare velden en onderlinge relaties. Zo kan informatie over een klant in een CRM-systeem anders worden voorgesteld dan in e-mails, documenten of agenda-items. Om resultaten uit verschillende bronnen op een zinvolle manier te combineren, is daarom een vorm van abstractie of normalisatie nodig.

## 3.3 Beperkingen van klassieke zoekmethoden

Een eerste beperking is dat klassieke zoekmethoden vaak sterk afhankelijk zijn van expliciete trefwoorden of gestructureerde filters. Daardoor hangt de kwaliteit van het resultaat in belangrijke mate af van de formulering van de gebruiker en van de zoekmogelijkheden die het onderliggende systeem aanbiedt. Wanneer andere terminologie, synoniemen of contextueel verwante begrippen worden gebruikt, bestaat het risico dat relevante resultaten niet worden gevonden. Vooral bij complexere of minder exact geformuleerde informatiebehoeften schieten zulke zoekmethoden daarom vaak tekort.

Daarnaast zijn veel traditionele zoekfuncties beperkt tot één afzonderlijk systeem. Een gebruiker kan bijvoorbeeld zoeken binnen e-mail, binnen een documentplatform of binnen een CRM-toepassing, maar niet op een uniforme manier over deze bronnen heen. Dit maakt het zoekproces traag, omslachtig en foutgevoelig.

Een bijkomende beperking is dat klassieke zoekmethoden meestal gericht zijn op het teruggeven van losse resultaten, eerder dan op het formuleren van een samengesteld antwoord. In veel praktijksituaties volstaat het echter niet om enkel een lijst van documenten of records te tonen. Gebruikers zoeken vaak een concreet antwoord op

een vraag waarvoor informatie uit meerdere bronnen gecombineerd moet worden.

## 3.4 Risico's van LLM-gebaseerde systemen

Hoewel LLM-gebaseerde systemen nieuwe mogelijkheden bieden voor het interpreteren van natuurlijke taal en het samenbrengen van informatie uit verschillende bronnen, brengen zij ook een aantal belangrijke risico's en beperkingen met zich mee. Binnen een bedrijfscontext wegen deze risico's extra zwaar, aangezien antwoorden betrouwbaar en controleerbaar moeten zijn en bestaande beveiligings- en toegangsregels gerespecteerd moeten worden.

Een eerste belangrijk risico is dat een LLM foutieve of verzonnen informatie kan genereren, ook wanneer het antwoord overtuigend en grammaticaal correct geformuleerd is. Deze hallucinaties maken dat de output van een model niet automatisch als feitelijk correct mag worden beschouwd.

Daarnaast is de output van een LLM probabilistisch van aard. Het model genereert geen deterministisch resultaat, maar genereert tekst op basis van waarschijnlijkheden. Daardoor kan eenzelfde vraag in verschillende gevallen tot een verschillend antwoord leiden.

Een ander belangrijk aandachtspunt betreft privacy en vertrouwelijkheid van gegevens. Wanneer LLM-gebaseerde systemen toegang krijgen tot interne bedrijfsinformatie, moet strikt bewaakt worden welke data aan het model wordt doorgestuurd. In een enterprise omgeving is het essentieel dat een dergelijk systeem geen informatie verwerkt of teruggeeft waarvoor de gebruiker geen rechten heeft.

Ook op beveiligingsvlak brengen LLM-systemen bijkomende risico's mee. Zo kunnen ze gevoelig zijn voor prompt injection, waarbij kwaadaardige instructies in externe inhoud proberen het gedrag van het model te beïnvloeden.

Deze risico's tonen aan dat LLM-gebaseerde systemen niet zomaar als autonome en volledig betrouwbare zoek- of antwoordsystemen mogen worden ingezet. Ze vereisen bijkomende maatregelen rond validatie, bronvermelding, toegangscontrole, contextselectie en beveiliging.

## 3.5 Functionele en niet-functionele vereisten

### 3.5.1 Functionele vereisten

Een eerste functionele vereiste is dat het systeem informatie moet kunnen ophalen uit meerdere, onderling verschillende databronnen. De oplossing moet in staat zijn om meerdere bronnen te raadplegen en de opgehaalde resultaten samen te brengen tot één bruikbaar geheel.

Het systeem moet natuurlijke taal van gebruikers kunnen verwerken. Gebruikers formuleren hun vragen vaak

niet als exacte zoektermen of gestructureerde filters, maar als een vraag of beschrijving. De oplossing moet dus overweg kunnen met dergelijke invoer en deze kunnen omzetten naar relevante acties binnen de onderliggende systemen.

Een derde functionele vereiste is dat het systeem kan bepalen welke databron of combinatie van databronnen nodig is om een vraag te beantwoorden. Wanneer een vraag informatie uit meerdere systemen vereist, moet de oplossing de juiste componenten aanspreken en de verkregen resultaten verder verwerken.

Ten slotte moet het systeem antwoorden of resultaten kunnen presenteren op een manier die begrijpelijk en bruikbaar is voor de eindgebruiker. In plaats van louter ruwe technische output of losse records terug te geven, moet de oplossing de gevonden informatie op een overzichtelijke manier ontsluiten.

### 3.5.2 Niet-functionele vereisten

Naast de functionele werking moet de oplossing ook voldoen aan een aantal niet-functionele vereisten. Een eerste belangrijke vereiste is veiligheid. Aangezien het systeem toegang krijgt tot interne bedrijfsdata, moet het bestaande authenticatie- en autorisatiemechanismen respecteren.

Een tweede belangrijke vereiste betreft privacy en vertrouwelijkheid. Het systeem moet kunnen omgaan met gevoelige informatie en mag enkel die gegevens verwerken die noodzakelijk zijn om een relevante zoekopdracht of antwoord te ondersteunen.

Verder is betrouwbaarheid een cruciale vereiste. De oplossing moet antwoorden zoveel mogelijk baseren op effectief opgehaalde gegevens uit de onderliggende systemen en mag niet uitsluitend steunen op gegenereerde output zonder duidelijke basis in brondata.

Ook uitbreidbaarheid vormt een belangrijke niet-functionele vereiste. Aangezien er steeds meer verschillende data aanwezig zijn binnen een bedrijf, moet de architectuur voldoende modulair zijn om in de toekomst bijkomende databronnen, tools of agents te kunnen integreren.

Daarnaast is gebruiksvriendelijkheid van belang. Het systeem moet een eenvoudige en uniforme interactiewijze aanbieden, ook wanneer de onderliggende technische werking uit meerdere componenten en databronnen bestaat.

Tot slot is ook performantie relevant. Hoewel niet elke zoekopdracht ogenblikkelijk beantwoord hoeft te worden, moet de responstijd voldoende beperkt blijven om de oplossing praktisch inzetbaar te maken in een dagelijkse werkomgeving.

# 4

# Architectuurontwerp

## 4.1 Overzicht van de voorgestelde architectuur

Om aan deze ontwerpdoelen te voldoen werd gekozen voor een multi-agentarchitectuur met een centrale orchestrator en meerdere gespecialiseerde subagents. Deze orchestrator vormt het centrale coördinatiepunt van het systeem. Hij ontvangt de gebruikersquery, interpreteert de vraag, bepaalt welke agent of combinatie van agents relevant is.

Voor elke geïntegreerde databron wordt een afzonderlijke agent voorzien. In het prototype gaat het om een Microsoft Graph agent, een Salesforce-Agent en een SmartSales-Agent. Elke agent bundelt de logica die nodig is om met één specifiek systeem te communiceren, inclusief kennis over de relevante APIs, querymogelijkheden, authenticatie en systeem-specifieke datamodellen.

De communicatie tussen agents en databronnen verloopt via een MCP-gebaseerde integratielaag. Dit maakt het mogelijk om de agentlogica en de externe systemen los te koppelen, wat de onderhoudbaarheid en uitbreidbaarheid van het systeem vergemakkelijkt.

Het globale systeem kan worden opgevat als een gelaagde architectuur. Bovenaan bevindt zich de gebruikersinteractie via natuurlijke taal. Daaronder bevindt zich de orchestrator, die instaat voor interpretatie en coördinatie. Vervolgens zijn er de gespecialiseerde agents, die elk via MCP en onderliggende API-koppelingen communiceren met hun respectieve databronnen. De opgehaalde resultaten worden teruggestuurd naar de orchestrator, die deze samenbrengt en omzet in een bruikbaar antwoord voor de gebruiker. Figuur 4.1 geeft een schematisch overzicht van deze architectuur.

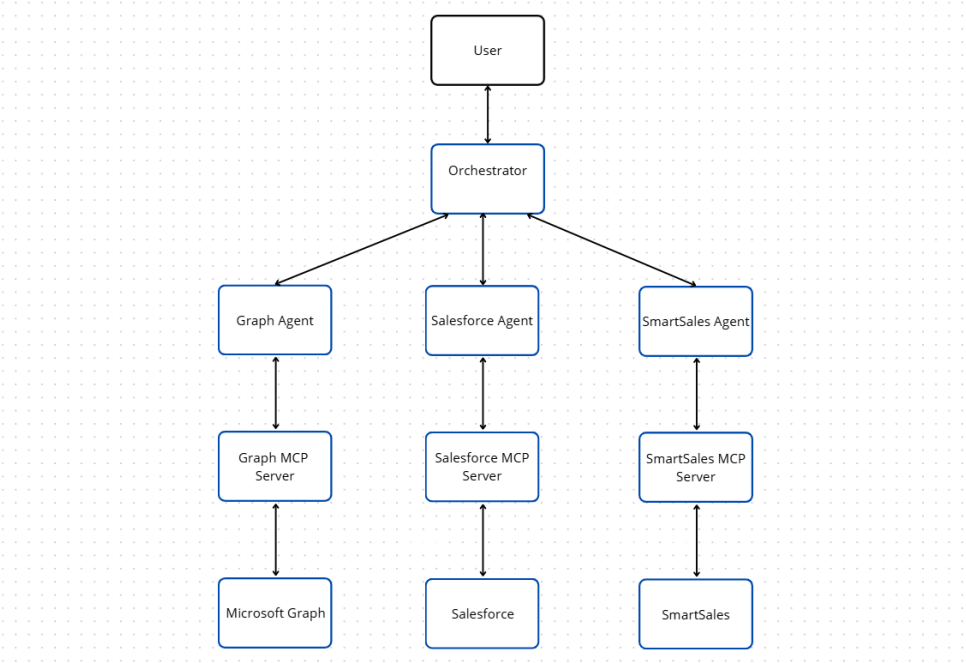


Figure 4.1: Overzicht van de voorgestelde multi-agentarchitectuur: de orchestrator coördineert drie gespecialiseerde subagents (Graph Agent, Salesforce-Agent en SmartSales-Agent), die elk via een afzonderlijke MCP-server communiceren met hun onderliggende databron.

## 4.2 Beschrijving van de componenten

### 4.2.1 Orchestrator

De orchestrator vormt het centrale coördinatiepunt van de voorgestelde architectuur. Hij ontvangt de natuurlijke taalquery van de gebruiker en stuurt het verdere zoekproces aan.

De orchestrator interpreteert de gebruikersvraag en bepaalt welke gespecialiseerde agent of combinatie van agents nodig is om deze te beantwoorden. Bij eenvoudige zoekopdrachten volstaat één agent, terwijl bij complexere vragen meerdere agents na elkaar of in combinatie kunnen worden aangesproken.

Daarnaast verzamelt de orchestrator de resultaten van de betrokken agents en brengt hij deze samen tot één antwoord dat aansluit bij de oorspronkelijke vraag van de gebruiker. De orchestrator is dus verantwoordelijk voor query-interpretatie, routing, coördinatie en antwoordsynthese, maar bevat zelf geen systeem-specifieke integratielogica.

### 4.2.2 Gespecialiseerde agents

Voor elke geïntegreerde databron wordt een afzonderlijke gespecialiseerde agent voorzien. In het prototype gaat het om een Microsoft Graph-Agent, een Salesforce-Agent en een SmartSales-Agent.

Deze opsplitsing is nodig omdat de betrokken systemen sterk verschillen in APIs, authenticatie, zoekmogelijkheden en datamodellen. Door deze systeem-specifieke complexiteit onder te brengen in afzonderlijke agents, blijft de rest van de architectuur beter afgebakend en onderhoudbaar.

Een gespecialiseerde agent ontvangt opdrachten van de orchestrator, zet deze om naar concrete acties binnen het gekoppelde systeem, en verwerkt de opgehaalde resultaten voor terugkoppeling naar de orchestrator. Deze aanpak maakt het bovendien mogelijk om later bijkomende databronnen te integreren door nieuwe agents toe te voegen, zonder de globale architectuur fundamenteel te wijzigen.

### 4.2.3 MCP-server of toollaag per databron

Tussen de gespecialiseerde agents en de onderliggende databronnen bevindt zich een MCP-gebaseerde serverlaag. Per databron wordt een afzonderlijke MCP-server voorzien die concrete operaties aanbiedt in de vorm van tools, zoals zoekopdrachten of het ophalen van specifieke gegevens.

Wanneer een agent een bepaalde actie uitvoert, gebeurt dit via een toolaanroep naar de overeenkomstige MCP-server. Deze server vertaalt de oproep vervolgens naar de juiste API-calls of systeemoperaties binnen het gekoppelde systeem. De concrete integratielogica bevindt zich dus in deze laag, waardoor de agentlogica en de systeemintegraties van elkaar gescheiden blijven.

### 4.2.4 Onderliggende databronnen

De voorgestelde architectuur steunt op bestaande bedrijfsapplicaties die elk hun eigen gegevens en functionaliteiten beheren. In het uitgewerkte prototype gaat het om Microsoft Graph, Salesforce en SmartSales.

Microsoft Graph fungeert als toegangspoort tot gegevens binnen het Microsoft 365-ecosysteem, zoals e-mails, agenda-items, contacten, bestanden en informatie uit toepassingen zoals Outlook, Teams, OneDrive en SharePoint. Salesforce wordt gebruikt als CRM-gerichte databron voor onder meer klanten, contactpersonen en opportuniteiten. SmartSales vormt een derde databron en bevat aanvullende salesgerelateerde informatie die niet rechtstreeks in Microsoft Graph of Salesforce beschikbaar is.

Een belangrijk uitgangspunt is dat deze onderliggende systemen zelf de *source of truth* blijven. De voorgestelde oplossing kopieert of centraliseert de volledige bedrijfsdata niet in één aparte databank, maar functioneert als

een zoek- en orchestratielaag bovenop de bestaande systemen.

## 4.3 Communicatie- en queryflow

De verwerking van een gebruikersvraag verloopt in de voorgestelde architectuur in een aantal opeenvolgende stappen. Eerst formuleert de gebruiker een query in natuurlijke taal. Deze query wordt ontvangen door de orchestrator, die optreedt als centraal coördinatiepunt van het systeem.

Vervolgens interpreteert de orchestrator de gebruikersvraag en bepaalt hij met ondersteuning van een LLM welke gespecialiseerde agent of combinatie van agents nodig is om de vraag te beantwoorden. Op basis daarvan wordt de query, eventueel in aangepaste vorm, doorgestuurd naar de betrokken agents.

Elke gespecialiseerde agent verwerkt de ontvangen opdracht binnen de context van zijn eigen databron. Om gegevens op te halen, communiceert de agent via de overeenkomstige MCP-server met het onderliggende systeem. De MCP-server vertaalt de toolaanroep naar een concrete API-oproep of systeemoperatie.

De opgehaalde resultaten worden vervolgens via dezelfde weg teruggestuurd naar de agent, die ze doorstuurt naar de orchestrator. Wanneer meerdere agents betrokken zijn, verzamelt de orchestrator de resultaten uit de verschillende databronnen en brengt hij deze samen tot één geïntegreerd antwoord voor de gebruiker.

Bij eenvoudige queries blijft deze flow beperkt tot één gespecialiseerde agent en één databron. Bij complexere queries kan dezelfde flow zich uitbreiden over meerdere agents en databronnen. Figuur 4.2 illustreert deze verwerking schematisch.

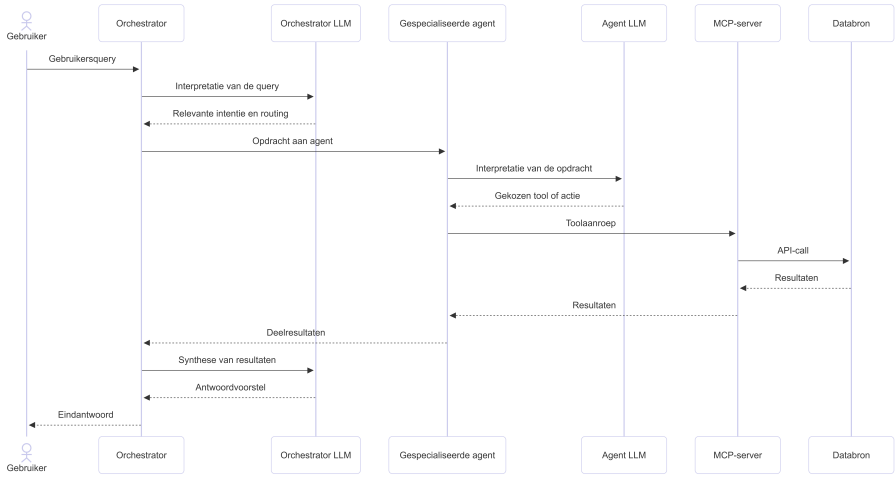


Figure 4.2: Schematische voorstelling van de communicatie- en queryflow: de gebruikersvraag wordt ontvangen door de orchestrator, die de relevante agents aanspreekt via de bijhorende MCP-servers. De deelresultaten worden samengevoegd tot één eindantwoord voor de gebruiker.

## 4.4 Veiligheids- en toegangsmodel

Aangezien de voorgestelde architectuur toegang biedt tot interne bedrijfsdata, vormt veiligheid een essentieel onderdeel van het ontwerp. De oplossing is daarom opgebouwd als een gecontroleerde laag bovenop bestaande bedrijfsapplicaties, waarbij de authenticatie- en autorisatiemechanismen van de onderliggende systemen behouden blijven.

Toegang tot databronnen zoals Microsoft Graph, Salesforce en SmartSales verloopt via de beveiligingsmechanismen die door deze systemen zelf worden voorzien, bijvoorbeeld via OAuth 2.0, access tokens of andere systeem-specifieke authenticatiestromen. De architectuur verleent dus geen autonome toegang tot data, maar steunt op de rechten en toegangsmechanismen die reeds in de bronsystemen aanwezig zijn.

Een belangrijk uitgangspunt daarbij is dat de zoekoplossing geen informatie mag teruggeven waarvoor de gebruiker in het oorspronkelijke systeem geen toegangsrechten heeft. De agents en de MCP-servers moeten deze beperkingen respecteren en mogen bestaande autorisatieregels niet omzeilen.

Daarnaast wordt de toegang tot bedrijfsdata functioneel afgebakend via gespecialiseerde agents en MCP-tools. Het taalmodel communiceert niet rechtstreeks met de onderliggende APIs, maar kan enkel gebruikmaken van de operaties die via de voorziene agents en toollagen beschikbaar worden gesteld.

## 4.5 Verantwoording van de architecturale keuzes

Bij het ontwerp van de voorgestelde architectuur werd gekozen voor een multi-agentaanpak met een centrale orchestrator en gespecialiseerde subagents per databron. Die keuze is in de eerste plaats ingegeven door de heterogeniteit van de betrokken systemen. Microsoft Graph, Salesforce en SmartSales verschillen sterk in termen van APIs, authenticatie, zoekmogelijkheden en datamodellen. Door per databron een afzonderlijke agent te voorzien, blijft deze systeem-specifieke complexiteit lokaal afgebakend en wordt de architectuur beter onderhoudbaar en uitbreidbaar.

Daarnaast werd gekozen voor een centrale orchestrator om de interactie met de gebruiker te vereenvoudigen. De gebruiker moet niet zelf bepalen welke databron geraadpleegd moet worden, maar kan één vraag in natuurlijke taal formuleren. De orchestrator interpreteert deze vraag en bepaalt welke agent of combinatie van agents nodig is om tot een antwoord te komen. Op die manier wordt de routinglogica centraal georganiseerd en kunnen resultaten uit meerdere systemen samengebracht worden.

Ten slotte werd een MCP-gebaseerde integratielaag gebruikt om de agentlogica los te koppelen van de concrete systeemintegraties. Daardoor blijven de onderliggende databronnen de source of truth, terwijl de architectuur toch voldoende modulair blijft om later bijkomende databronnen of tools toe te voegen.

# 5

# Implementatie

## 5.1 Gebruikte technologieën en stack

De implementatie van de eerste versie van het prototype volgt de architectuur uit hoofdstuk 4. Centraal daarin staan een orchestrator, meerdere gespecialiseerde agents en per databron een bijhorende MCP-server. Binnen deze MCP-servers wordt de logica ondergebracht voor de communicatie met externe systemen, inclusief de concrete toolimplementaties, de interactie met de onderliggende APIs en de afhandeling van authenticatie.

Als basis voor de agentlogica werd gebruikgemaakt van het Microsoft Agent Framework. Dit framework wordt gebruikt voor het definiëren van de agents, het registreren van tools, het beheren van sessies en het lokaal testen van de applicatie via de DevUI, de ingebouwde debugomgeving van het Microsoft Agent Framework. Voor de onderliggende taalmodellen wordt Azure OpenAI gebruikt, aangesproken via de Python SDK van OpenAI.

Voor de koppeling met externe systemen werd gebruikgemaakt van MCP. Voor elk geïntegreerd systeem werd een afzonderlijke MCP-server uitgewerkt, namelijk voor Microsoft Graph, Salesforce en SmartSales. Deze servers stellen systeemgebonden functionaliteit beschikbaar in de vorm van tools die door de agents kunnen worden opgeroepen. De MCP-servers zijn opgebouwd met FastMCP en worden aangeboden als asynchrone HTTP-services. De beschikbare tools worden extern beschreven in YAML-bestanden die bij het opstarten worden ingelezen.

Voor de koppeling met Microsoft 365 wordt gebruikgemaakt van de officiële Microsoft Graph SDK. Voor authenticatie richting Microsoft Graph worden onder meer `msal` en `azure-identity` gebruikt. Voor Salesforce werd geen aparte SDK gebruikt, maar wordt rechtstreeks gewerkt met de Salesforce REST API via `httpx`. Voor datamodellering wordt gebruikgemaakt van Pydantic-modellen, zodat gegevens op een consistente manier tussen de

verschillende lagen van het systeem kunnen worden uitgewisseld.

## 5.2 Implementatie van de orchestrator

Het gedrag van de orchestrator wordt voornamelijk bepaald door de systeemprompt, namelijk de vaste instructies die de rol, verantwoordelijkheden en het toegelaten toolgebruik van de agent beschrijven.

De orchestrator werkt volgens een hiërarchisch model waarbij de Graph-Agent, Salesforce-Agent en Smartsales-Agent als oproepbare tools aan hem worden aangeboden. Voor elke gespecialiseerde agent is daarbij een wrapper voorzien die een asynchrone oproep uitvoert naar de overeenkomstige agent en het resultaat teruggeeft aan de orchestrator.

De routing van gebruikersvragen gebeurt in deze eerste versie volledig via het taalmodel. Er is geen aparte rule-based classifier of expliciete intentiedetectielaag aanwezig. Op basis van de inhoud van de gebruikersvraag bepaalt de orchestrator welke agent moet worden aangesproken. Bij queries die informatie uit meerdere systemen vereisen, kan de orchestrator meerdere agents na elkaar oproepen en hun resultaten samenbrengen.

De orchestrator is in deze eerste versie bewust beperkt gehouden tot een lichte coördinatielaag. Hij bevat zelf geen systeemspecifieke integratielogica en houdt ook geen uitgebreide gedeelde conversatiestatus bij tussen de verschillende gespecialiseerde agents. Deze eerste versie volstaat om de voorgestelde architectuur werkend aan te tonen, maar laat nog ruimte voor verdere uitbreiding.

## 5.3 Implementatie van de Microsoft Graph-Agent

De Microsoft Graph-Agent is verantwoordelijk voor vragen die betrekking hebben op gegevens binnen Microsoft 365. In de huidige implementatie ondersteunt hij onder meer e-mails, bestanden, contacten en agenda-items.

De agent is opgebouwd binnen het Microsoft Agent Framework en gebruikt een Azure OpenAI-model om opdrachten van de orchestrator te interpreteren, de juiste tool te selecteren en de opgehaalde resultaten verder te verwerken. De feitelijke toegang tot Microsoft 365 verloopt via een gekoppelde Graph MCP-server.

Binnen deze MCP-server wordt de systeemspecifieke logica afgehandeld. De beschikbare operaties omvatten onder meer het opvragen van e-mails, het zoeken naar bestanden, het uitlezen van documenten, het raadplegen van contacten en het opvragen van kalenderinformatie. Deze operaties worden via configuratie geregistreerd, zodat de beschikbare tools losgekoppeld blijven van de agentimplementatie.

Voor de communicatie met Microsoft Graph wordt gebruikgemaakt van een repositorylaag op basis van de officiële Microsoft Graph SDK. Deze laag vertaalt toolaanroepen naar concrete aanvragen naar Microsoft 365 en zet de ontvangen resultaten om naar gestructureerde objecten.

De authenticatie verloopt via gedelegeerde toegang. De Graph-aanroepen worden uitgevoerd in naam van de geauthenticeerde gebruiker, zodat de agent enkel toegang heeft tot gegevens waarvoor die gebruiker binnen Microsoft 365 gemachtigd is.

In deze eerste versie werd daarnaast een beperkte vorm van sessiecontext voorzien voor bestandsgerelateerde interacties. Wanneer tijdens een sessie documenten worden gevonden, kan de agent bepaalde informatie daarover tijdelijk bijhouden om vervolgvragen beter te ondersteunen.

Aangezien de zoekfunctionaliteit van Microsoft Graph voor bestanden en e-mails voornamelijk keyword-gebaseerd is, werd daarnaast ook al een eerste semantische zoekaanpak voor documenten uitgewerkt op basis van vector embeddings. Daardoor kan niet alleen gezocht worden op bestandsnaam of metadata, maar ook op de inhoud en betekenis van documenten.

## 5.4 Implementatie van de Salesforce-Agent

De Salesforce-Agent is verantwoordelijk voor vragen over CRM-gegevens, zoals accounts, contacten, leads, opportunities en cases. Net zoals de Graph-Agent werd ook deze agent geïmplementeerd binnen het Microsoft Agent Framework en gekoppeld aan een Azure OpenAI-model.

De feitelijke toegang tot Salesforce verloopt via een afzonderlijke MCP-server, die de beschikbare Salesforce-functionaliteit ter beschikking stelt in de vorm van tools. Deze tools worden gedefinieerd in een YAML-bestand en bij het opstarten van de MCP-server geregistreerd.

Achter de MCP-server bevindt zich een repositorylaag die instaat voor de effectieve communicatie met Salesforce via de Salesforce REST API. In deze laag worden SOQL-queries opgebouwd en uitgevoerd om de nodige gegevens op te halen.

De authenticatie voor Salesforce gebeurt via een sessiegebaseerde aanpak op het niveau van de MCP-server. In de implementatie zijn twee authenticatiestromen aanwezig. Enerzijds is er een interactieve OAuth Authorization Code Flow, die via de browser gebruikt kan worden. Anderzijds is er ook ondersteuning voor een JSON Web Token (JWT)-gebaseerde OAuth-flow voor niet-interactieve server-to-server scenario's. In de huidige implementatie wordt deze JWT-gebaseerde flow ook als standaardflow gebruikt in de geautomatiseerde authenticatielogica.

De Salesforce-Agent is in deze eerste versie stateless opgevat. In tegenstelling tot de Graph-Agent is er geen

bijkomende contextprovider voorzien die informatie uit eerdere interacties binnen dezelfde sessie bewaart.

## 5.5 Implementatie van de SmartSales-Agent

De SmartSales-Agent is verantwoordelijk voor vragen over verkoopgerelateerde gegevens, zoals locaties, catalogusartikelen en bestellingen. Net zoals de Graph- en Salesforce-Agent werd ook deze agent geïmplementeerd binnen het Microsoft Agent Framework en gekoppeld aan een Azure OpenAI-model.

De toegang tot SmartSales verloopt via een afzonderlijke MCP-server, die de beschikbare SmartSales-functionaliteit ter beschikking stelt in de vorm van tools. Deze tools worden gedefinieerd in een YAML-bestand en bij het opstarten van de MCP-server geregistreerd. De beschikbare operaties zijn georganiseerd rond drie domeinen: gebruikers, catalogus en bestellingen.

Achter de MCP-server bevindt zich een repositorylaag die instaat voor de effectieve communicatie met de SmartSales REST API. In tegenstelling tot de Graph-Agent, waarvoor een officiële SDK beschikbaar is, wordt hier gebruikgemaakt van een directe asynchrone HTTP-client. De queryparameters worden daarbij rechtstreeks doorgegeven aan de SmartSales-API, die een eigen querynotatie hanteert.

Om de kwaliteit van toolaanroepen te verbeteren, worden bij het opstarten van de MCP-server de beschikbare filter-, sorteer- en weergavevelden voor alle domeinen opgehaald en lokaal gecachet. Bij elke toolaanroep worden de opgegeven filter- en sorteervelden gevalideerd aan de hand van deze cache, zodat ongeldige parameters vroegtijdig worden onderschept en de agent een duidelijke foutmelding ontvangt.

De authenticatie voor SmartSales verloopt in deze versie volledig niet-interactief. De MCP-server authenticeert zich bij het opstarten automatisch via omgevingsvariabelen, zonder tussenkomst van een gebruiker via de browser. Wanneer een toegangstoken verloopt, wordt de authenticatie automatisch opnieuw uitgevoerd op basis van dezelfde configuratie. Voor de opslag van tokens is een bestandsgebaseerde token store voorzien.

Net als de Salesforce-Agent is ook de SmartSales-Agent in deze eerste versie stateless opgevat. Er is geen contextprovider voorzien die informatie uit eerdere interacties binnen dezelfde sessie bewaart.

## 5.6 Integratie tussen agents en databronnen

De integratie tussen de verschillende lagen van het systeem volgt een vast patroon. De orchestrator ontvangt een gebruikersquery en roept, indien nodig, één of meerdere gespecialiseerde agents aan. Elke gespecialiseerde agent selecteert vervolgens een geschikte tool op zijn gekoppelde MCP-server. De MCP-server vertaalt deze toolaanroep naar een concrete interactie met het onderliggende systeem en geeft het resultaat terug via dezelfde keten.

Deze opbouw zorgt ervoor dat de verantwoordelijkheden duidelijk gescheiden blijven. De orchestrator behandelt

de coördinatie, de gespecialiseerde agents verzorgen interpretatie en toolselectie binnen hun eigen domein, en de MCP-servers kapselen de systeemspecifieke integratielogica af. Daardoor kan een nieuwe databron worden toegevoegd zonder de globale interactiestroom fundamenteel te wijzigen.

## 5.7 Beperkingen van de huidige implementatie

Hoewel het prototype de voorgestelde architectuur functioneel aantoont, blijft de huidige implementatie op verschillende punten bewust beperkt. Een eerste beperking is dat de routing in de orchestrator volledig via het taalmodel gebeurt en dus niet ondersteund wordt door een afzonderlijke deterministische classificatielaag.

Daarnaast is contextbeheer voorlopig slechts beperkt uitgewerkt. De orchestrator houdt geen gedeelde sessiestatus bij over meerdere gespecialiseerde agents heen, en ook binnen de agents is contextondersteuning slechts gedeeltelijk aanwezig.

Verder is de foutafhandeling nog beperkt. Scenario's zoals time-outs, mislukte toolaanroepen of gedeeltelijke resultaten bij onbeschikbare databronnen worden in deze versie nog niet volledig robuust afgehandeld. Ook multi-user ondersteuning en productiegerichte deployment zijn nog slechts gedeeltelijk uitgewerkt.

De huidige implementatie moet daarom in de eerste plaats worden beschouwd als een functioneel prototype dat de haalbaarheid van de architectuur aantoont, en niet als een volledig productieklare oplossing.

# 6

# Evaluatiemethodologie

## 6.1 Evaluatie-opzet

Dit hoofdstuk evalueert het voorgestelde multi-agentsysteem op basis van vier dimensies: correctheid van het gegenereerde antwoord, correctheid van de query routing, responstijd en tokenverbruik. Deze dimensies sluiten aan bij de centrale doelstelling van deze masterproef, namelijk nagaan in welke mate een multi-agentarchitectuur geschikt is om vragen van gebruikers correct te interpreteren, naar de juiste databronnen te routeren en op een praktisch bruikbare manier te beantwoorden.

De evaluatie wordt uitgevoerd op het uitgewerkte prototype. Daarbij wordt zowel gekeken naar vragen die door één enkele agent kunnen worden afgehandeld als naar vragen waarvoor meerdere agents moeten samenwerken. Op die manier wordt niet alleen de kwaliteit van de individuele subagents beoordeeld, maar ook de werking van de orchestrator als coördinerende component.

De correctheid van de gegenereerde antwoorden wordt niet handmatig voor alle queries beoordeeld, maar via een afzonderlijke LLM-evaluator. Het gebruik van een taalmodel als evaluator voor open-tekst antwoorden, ook wel *LLM-as-a-judge* genoemd, is een gevestigde aanpak in de evaluatie van generatieve systemen [16]. Deze evaluator vergelijkt het gegenereerde antwoord met een vooraf opgesteld referentieantwoord en kent op basis daarvan een score toe. Deze aanpak is gekozen omdat de output van het systeem uit vrije tekst bestaat en de inhoud van de onderliggende databronnen kan veranderen. Daardoor is een eenvoudige exacte vergelijking met één vast referentieantwoord in veel gevallen niet geschikt.

De query routing wordt geëvalueerd door per testquery na te gaan of de orchestrator de verwachte agent of combinatie van agents heeft aangesproken. De performantie van het systeem wordt uitgedrukt in responstijd, gemeten tussen het indienen van de vraag en het ontvangen van het volledige antwoord. De kost van het systeem wordt benaderd via het tokenverbruik van de gebruikte LLM-aanroepen.

Er werd gekozen voor `gpt-4o-mini` als evaluator omdat dit model voor deze benchmark een praktisch evenwicht biedt tussen beoordelingskwaliteit en kost, en daardoor geschikt is voor het automatisch scoren van een

grotere set testqueries.

## 6.2 Testscenario's en querycategorieën

### 6.2.1 Samenstelling van de testset

De testset van 88 queries werd samengesteld via een combinatie van AI-ondersteuning en handmatige selectie. Een taalmodel werd ingezet om initiële querysuggesties te genereren per subcategorie en databron. De uiteindelijke queries werden vervolgens handmatig geselecteerd en bijgesteld, zodat ze representatief zijn voor realistische vragen en voldoende spreiding vertonen over de functionaliteit van elke agent.

Aan elke query werd handmatig een moeilijkheidsgraad toegewezen: *simple*, *medium* of *hard*. Eenvoudige queries vereisen een directe opvraging zonder bijzondere filtercriteria. Queries van gemiddelde moeilijkheid bevatten filtercondities, specifieke veldselectie of een zoekopdracht met meerdere parameters. Moeilijke queries vereisen meerdere opeenvolgende toolaanroepen, het doorgeven van resultaten tussen stappen, of een complexe interpretatie van de vraag.

Om de evaluatie reproduceerbaar en consistent uit te voeren, werd een script gebruikt dat alle testqueries automatisch indient en de resultaten vastlegt. De evaluatie focust daarbij in de eerste plaats op de technische werking van het systeem.

### 6.2.2 Overzicht van de querycategorieën

De testset is onderverdeeld in vier groepen, één per geëvalueerde agent. Elke groep bestaat uit meerdere subcategorieën die de functionaliteit van de betrokken databron gebruikt. Tabel 6.1 geeft een overzicht van de verdeling.

De queries voor de GraphAgent, SalesforceAgent en SmartSalesAgent zijn opgebouwd rond de objecttypen van de databron. Voor de GraphAgent betreft dit de voornaamste entiteiten binnen Microsoft 365: e-mailberichten, agenda-items, contacten, gebruikersinformatie en bestanden in OneDrive. Voor de SalesforceAgent worden de centrale CRM-objecten bevraagd: accounts, contacten, leads, opportunities en cases. De SmartSalesAgent wordt geëvalueerd op queries over locaties, catalogusitems en orders, inclusief vragen over de beschikbare filtervelden en projectiemogelijkheden die eigen zijn aan die API.

De queries voor de OrchestratorAgent zijn in twee categorieën verdeeld. Routingqueries zijn enkelvoudige vragen gericht op precies één databron en dienen om na te gaan of de orchestrator de juiste subagent selecteert. Cross-system queries vereisen informatie uit meerdere databronnen tegelijk en evalueren de capaciteit van de orchestrator om subagents te combineren en resultaten samen te voegen tot één coherent antwoord.

De volledige lijst van alle 88 testqueries is opgenomen in bijlage 8.6.

## 6.3 Evaluatie van correctheid

De correctheid van het systeem wordt hier gezien als de mate waarin het gegenereerde antwoord inhoudelijk overeenkomt met wat er verwacht wordt. Correctheid is niet enkel feitelijke juistheid, maar ook volledigheid en relevantie.

Om de correctheid te beoordelen wordt er voor elke testquery informatie gegeven over wat precies verwacht wordt. Dit bevat niet noodzakelijk een volledig uitgeschreven respons, maar beschrijft welke informatie minimaal aanwezig moet zijn om het antwoord als correct te kunnen beschouwen.

Een eenvoudige juist/fout-beoordeling volstaat hier niet. Met een schaal van 1 tot 5 kan de nuance beter worden vastgelegd. Een score van 1 betekent dat het antwoord fout, leeg of volledig irrelevant is. Een score van 2 duidt op een duidelijk ontoereikend antwoord. Een score van 3 staat voor een gedeeltelijk correct antwoord waarbij de kern van de vraag geraakt wordt, maar belangrijke informatie ontbreekt. Een score van 4 betekent dat het antwoord grotendeels correct en bruikbaar is, met enkel beperkte onnauwkeurigheden. Een score van 5 duidt op een volledig correct, relevant en voldoende volledig antwoord.

De resultaten van deze evaluatie worden op meerdere niveaus bekeken: per agent, per querycategorie en per type (single-agent versus multi-agent).

## 6.4 Evaluatie van query routing

Naast de inhoudelijke correctheid van de antwoorden wordt ook nagegaan of queries naar de juiste subagents worden gerouteerd. Hiervoor werd een observatielaag toegevoegd aan de orchestrator. Bij elke run wordt een routing trace gestart, waarin automatisch wordt bijgehouden welke subagents door de orchestrator worden aangesproken.

Per aanroep wordt onder meer opgeslagen welke agent werd gebruikt, in welke volgorde dit gebeurde, welke query werd doorgestuurd en of de aanroep succesvol was. De effectieve routing wordt vervolgens vergeleken met de vooraf verwachte routing.

Daarbij wordt onderscheid gemaakt tussen verschillende fouttypes. Bij under-routing ontbreekt een vereiste agent, bij over-routing wordt een extra niet-noodzakelijke agent aangesproken, en bij mis-routing wordt een

verkeerde agent gekozen.

## 6.5 Evaluatie van performantie en kost

### 6.5.1 Evaluatie van de responstijd

De performantie van het prototype wordt geëvalueerd op basis van de responstijd per query. Onder responstijd wordt hier de totale tijd verstaan tussen het starten van de verwerking van een query en het moment waarop het volledige antwoord is teruggegeven. Het gaat dus om een end-to-end meting.

De gemeten tijd bevat de volledige verwerking van het systeem, inclusief de inferentietijd van het taalmodel, de toolcalls, externe API-oproepen en het uiteindelijk samenstellen van de opgehaalde informatie tot één antwoord.

### 6.5.2 Evaluatie van het tokenverbruik

Naast responstijd wordt ook het tokenverbruik bijgehouden als benadering van de kost van het systeem. Per query worden drie waarden opgeslagen: het aantal inputtokens, het aantal outputtokens en het totaal aantal tokens. Inputtokens omvatten alles wat naar het model gestuurd wordt, zoals de systeemprompt, de vraag van de gebruiker, beschrijvingen van tools en eventuele toolresultaten. Outputtokens zijn de tokens van het uiteindelijke gegenereerde antwoord.

In de analyse wordt het tokenverbruik per query mee opgenomen en vergeleken tussen verschillende agent-modi en querytypes, zodat zichtbaar wordt welke vragen relatief goedkoop blijven en welke net veel context of meerdere agentaanroepen vereisen.

## 6.6 Beperkingen van de evaluatie

Hoewel deze evaluatie een duidelijk beeld geeft van de werking van het voorgestelde systeem, zijn er ook enkele beperkingen waarmee rekening moet worden gehouden.

Een eerste beperking is dat de correctheid van de antwoorden beoordeeld wordt met behulp van een afzonderlijk taalmodel. Die aanpak is nuttig om vrije tekst op een genuanceerde manier te beoordelen, maar blijft gedeeltelijk afhankelijk van dat model. Daardoor is volledige objectiviteit niet gegarandeerd.

Daarnaast is de evaluatie gebaseerd op een vooraf samengestelde set testqueries die beperkt is in omvang. De resultaten geven daarom vooral inzicht in de prestaties van het systeem binnen deze geselecteerde testgevallen.

Ook de gebruikte databronnen zijn dynamisch. Aangezien gegevens in Microsoft Graph, Salesforce en SmartSales

kunnen veranderen in de tijd, is het mogelijk dat dezelfde query op een ander moment een ander resultaat oplevert.

Verder is ook de evaluatie van query routing gebaseerd op een vooraf bepaalde verwachte routing per query. In de meeste gevallen is die duidelijk, maar bij complexere vragen kunnen soms meerdere routing strategieën verdedigbaar zijn.

Table 6.1: Overzicht van de testqueries per agent en subcategorie

| **Agent** | **Subcategorie** | **Aantal** |
|---|---|---|
| GraphAgent | Identity | 1 |
| | E-mail | 7 |
| | People | 2 |
| | Kalender | 4 |
| | Contacten | 2 |
| | Bestanden | 6 |
| | *Subtotaal* | *22* |
| SalesforceAgent | Accounts | 5 |
| | Contacts | 4 |
| | Leads | 4 |
| | Opportunities | 5 |
| | Cases | 4 |
| | *Subtotaal* | *22* |
| SmartSalesAgent | Locaties | 7 |
| | Catalogus | 6 |
| | Orders | 9 |
| | *Subtotaal* | *22* |
| OrchestratorAgent | Routing queries | 8 |
| | Cross-system queries | 14 |
| | *Subtotaal* | *22* |
| **Totaal** | | **88** |

# 7

# Evaluatieresultaten

## 7.1 Algemene resultaten

De evaluatie omvat 88 testqueries verdeeld over vier agenten, elk met precies 22 queries. Alle queries werden technisch succesvol uitgevoerd en er traden geen fatale systeemfouten op. De inhoudelijke kwaliteit verschilt echter sterk tussen de agenten.

Op het niveau van de gemiddelde LLM-score (schaal 1–5) presteert de SmartSalesAgent het sterkst met 4,18, gevolgd door de GraphAgent (3,82) en de OrchestratorAgent (3,77), en op afstand de SalesforceAgent (2,59). De orchestrator behandelt uitsluitend complexere vragen, namelijk enkelvoudige routingtests en vragen die antwoorden uit meerdere systemen vereisen. Deze scoort gemiddeld iets lager dan de GraphAgent, maar het verschil is beperkt. Dit suggereert dat de extra routinglaag weinig effect heeft op de antwoordkwaliteit.

Tabel 7.1 geeft een overzicht van de voornaamste resultaten per agent.

Table 7.1: Overzicht van evaluatieresultaten per agent (meest recente run, 31 maart 2026)

| Agent | n | Gem. score | Score $\geq$4 (%) | Gem. tijd (s) | Gem. inp. tok. | Gem. uit. tok. |
|---|---|---|---|---|---|---|
| GraphAgent | 22 | 3,82 | 72,7 | 14,6 | 6.430 | 975 |
| SalesforceAgent | 22 | 2,59 | 36,4 | 13,1 | 2.946 | 405 |
| SmartSalesAgent | 22 | 4,18 | 72,7 | 13,9 | 5.170 | 1.205 |
| OrchestratorAgent | 22 | 3,77 | 63,6 | 24,3 | 2.266 | 437 |

## 7.2 Evaluatie van correctheid

### GraphAgent

De GraphAgent behaalt over 22 queries een gemiddelde score van 3,82, met een mediane score van 4 en een standaardafwijking van 1,26. Eenvoudige vragen, zoals identiteitsvragen en het koppelen van mensen aan hun e-mailadres, scoren het hoogst. De e-mailqueries presteren goed met een gemiddelde van 4,14, maar de vraag die de volledige e-mailbody opvraagt scoort 1 en is tegelijk de traagste query in de volledige testset (102,78 seconden, zie §7.4.1). Vragen over specifieke bestandsinhoud en bestandstypen, zoals Excel-bestanden, scoren 2, terwijl zoekopdrachten in OneDrive goed verlopen.

Kalenderqueries presteren het zwakst met een gemiddelde van 3,00. Vaak treedt hetzelfde probleem op: de agent slaagt er niet in om de gevraagde tijdsperiode correct te interpreteren. Vragen naar gebeurtenissen binnen this week" of next week" worden soms beantwoord met events buiten de gevraagde periode, wat resulteert in scores van 2.

Opmerkelijk is dat de moeilijkheidsgraad nauwelijks verschil maakt: eenvoudige, gemiddelde en moeilijke queries scoren respectievelijk 3,86, 3,80 en 3,80. De kwaliteitsverschillen zijn dus niet te herleiden tot de taalcomplexiteit van de vraag, maar tot de aard van de onderliggende Microsoft Graph-endpoints en de volledigheid van de toolimplementatie.

### SalesforceAgent

De SalesforceAgent is duidelijk de zwakste schakel van het systeem. Over 22 queries bedraagt de gemiddelde score 2,59, met een mediane score van 2,5 en een standaardafwijking van 1,56. Elf van de 22 antwoorden scoren 1 of 2 (50,0%), waarvan negen met score 1. De problematiek is structureel: bij meerdere querytypes geeft de agent systematisch lege lijsten terug. Drie van de vier leadsqueries met inhoudelijke filteringscriteria retourneren een lege `actual_response`, net als de query naar opportunities in de *Proposal/Price Quote*-fase en meerdere queries naar open cases.

Wanneer de agent wél data retourneert, ontbreken regelmatig verwachte velden. Contactqueries leveren geen e-mailadressen, casesqueries missen velden als `CaseNumber` en `Status`, en de meest complexe accountquery, namelijk accounts in Frankrijk met gekoppelde contacten, geeft een leeg resultaat. Dit wijst in de eerste plaats op een onvolledig veldbeleid in de onderliggende toolconfiguratie: de SOQL-queries hanteren een te beperkte veldselectie en ondersteunen relationele queries onvoldoende. Tegelijk moet worden opgemerkt dat ook de gebruikte sandboxomgeving een beperkende factor vormt. Voor sommige vraagtypes is de beschikbare testdata te beperkt of te weinig representatief om een inhoudelijk volledig antwoord te kunnen formuleren, zelfs wanneer de query op zich correct is opgebouwd. Dit zal zeker meegenomen en bekeken worden in de tweede fase.

De *accounts*-categorie vormt een relatieve uitzondering met een gemiddelde score van 3,60 over vijf queries. De *leads*-categorie haalt het laagste gemiddelde (2,00), gevolgd door *cases* (2,00) en *contacts* (2,50). De moeilijkheidsgraad correleert hier wél met de kwaliteit: beide als *hard* geclassificeerde queries scoren 1, tegenover een gemiddelde van 2,54 voor *medium* en 3,14 voor *simple*.

### SmartSalesAgent

De SmartSalesAgent presteert het meest consistent van de drie subagenten, met een gemiddelde score van 4,18, een mediane score van 5 en slechts drie antwoorden met een score van 2. De *orders*-categorie is bijzonder sterk: acht van de negen queries scoren 4 of 5, en de gemiddelde score bedraagt 4,56. De *catalog*-categorie (gemiddeld 4,00 over zes queries) en de *locations*-categorie (3,86 over zeven queries) scoren iets lager, maar blijven solide.

Alle drie de lage scores (telkens 2) volgen hetzelfde patroon: multi-step queries waarbij de agent eerst een lijst ophaalt en vervolgens de volledige details van het eerste resultaat moet ophalen. In elk geval slaagt de agent er niet in om de uitvoer van de eerste toolaanroep correct door te geven als invoer voor de tweede. Dit is een beperking in de chaining-logica, niet in de toegang tot de API.

De SmartSalesAgent vertoont ook het sterkste verband tussen moeilijkheidsgraad en score: eenvoudige queries scoren gemiddeld 4,71 over veertien queries, terwijl moeilijke queries terugvallen op 2,75 over vier queries. Dit verschil is intuïtief verklaarbaar: complexere SmartSales-queries vereisen combinaties van toolaanroepen, doorgaans chaining-queries, die de agent nog niet betrouwbaar uitvoert.

### OrchestratorAgent

De OrchestratorAgent behaalt een gemiddelde score van 3,77 over 22 queries, met een mediane score van 4 en een standaardafwijking van 1,11. De querysamenstelling omvat acht enkelvoudige routingtests, gericht op één specifieke subagent, en veertien cross-system queries. De cross-system queries scoren gemiddeld 3,93, tegenover 3,50 voor de enkelvoudige routingtests.

Het lagere gemiddelde voor enkelvoudige routingtests is verklaarbaar: de kwaliteit van het antwoord is daar volledig afhankelijk van de prestaties van de onderliggende subagent. De twee routingtests gericht op de SalesforceAgent scoren respectievelijk 2 en 4; de GraphAgent-routingtest over een e-mailzoekopdracht scoort 2 ondanks correcte routing, doordat de agent de gevraagde e-mailinhoud niet volledig teruggeeft. Dit toont dat routingfouten en antwoordkwaliteit ontkoppeld zijn: ook een perfect gerouteerde query kan resulteren in een onvolledig antwoord.

De cross-system resultaten zijn inhoudelijk sterker: vijf van de veertien queries scoren 5, vier scoren 4. Queries die meerdere bronnen combineren, zoals recente e-mails koppelen aan Salesforce-contacten of agenda-items verbinden met Salesforce-accounts, worden goed afgehandeld wanneer de betrokken subagenten voldoende

data retourneren. De drie scores van 3 in de cross-system categorie zijn telkens te herleiden naar onvolledige Salesforce-responses die de vergelijkingslogica van de orchestrator tegenwerken.

## 7.3 Evaluatie van query routing

De routinganalyse is gebaseerd op de automatisch gegenereerde *routing traces* die per query bijhouden welke subagenten door de orchestrator worden aangesproken, in welke volgorde en met welk resultaat.

### Enkelvoudige routing

Voor de acht routingtests met enkelvoudige routing, namelijk *routing/graph*, *routing/salesforce* en *routing/smartsales*, bedraagt de nauwkeurigheid 100%: de orchestrator kiest in elk geval exact de verwachte subagent. Er werd geen enkel geval van mis-routing of under-routing vastgesteld. De orchestrator herkent op basis van de vraaginhoud correct het betrokken systeem, zowel voor Microsoft 365-, Salesforce- als SmartSales-queries.

Dat de routing correct is, impliceert niet dat de antwoorden ook kwalitatief zijn. Drie van de acht enkelvoudige routingtests scoren 2: twee Salesforce-routingtests en één Graph-routingtest. Dit patroon is consistent met de eerder vastgestelde prestatietekorten bij die subagenten.

### Cross-system routing

Bij de veertien cross-system queries gebruikt de orchestrator steeds meerdere subagenten. De meest voorkomende combinatie is *graph + salesforce* (vijf keer), gevolgd door *salesforce + smartsales* (drie keer) en *graph + salesforce + salesforce* (twee keer). In het laatste geval roept de orchestrator de SalesforceAgent tweemaal aan binnen dezelfde query, met telkens een verfijnde deelvraag gebaseerd op eerder verkregen informatie. Beide gevallen resulteren in een score van 5, wat aangeeft dat iteratieve subagentaanroepen functioneel zijn wanneer ze op de juiste informatiebehoefte zijn afgestemd.

Het enige geval van over-routing betreft de query *"Find emails I received this week and check if any senders are linked to a SmartSales location"*: de orchestrator roept de SmartSalesAgent driemaal aan naast één GraphAgent-aanroep, wat resulteert in een score van 2 en de op één na hoogste tokenuitgave van de Orchestrator-run. De herhaalde SmartSales-aanroepen zijn functioneel overbodig. Under-routing en mis-routing werden niet vastgesteld in de dataset.

Tabel 7.2 vat de routingresultaten samen.

Table 7.2: Routingresultaten per categorie (OrchestratorAgent, run 0f9f37a4)

| Categorie | n | Gem. score | Routing |
|---|---|---|---|
| routing/graph | 3 | 4,00 | 100% correct (3/3) |
| routing/salesforce | 2 | 3,00 | 100% correct (2/2) |
| routing/smartsales | 3 | 3,33 | 100% correct (3/3) |
| cross-system | 14 | 3,93 | n.v.t. (variabele verwachte routing) |

## 7.4 Evaluatie van performantie en kost

### 7.4.1 Evaluatie van de responstijd

De gemiddelde responstijden van de drie subagenten liggen dicht bij elkaar: 14,6 seconden voor de GraphAgent, 13,1 seconden voor de SalesforceAgent en 13,9 seconden voor de SmartSalesAgent. De OrchestratorAgent vereist met 24,3 seconden gemiddeld beduidend meer tijd, wat verwacht is gezien de opeenvolgende subagentaanroepen.

De spreiding is voor de GraphAgent het grootst: de standaardafwijking bedraagt 23,0 seconden bij een mediane responstijd van slechts 6,7 seconden. De overgrote meerderheid van queries wordt snel verwerkt, maar een klein aantal gevallen trekt het gemiddelde sterk omhoog. De piekwaarde van 102,8 seconden, voor de query naar de volledige e-mailbody, is het meest extreme geval en wijst op een toolaanroepcyclus waarbij de agent herhaaldelijk probeerde de inhoud op te halen. Dit geval vertegenwoordigt tevens het hoogste tokenverbruik van de volledige dataset (49.581 tokens). Op duidelijke afstand volgen twee zoekopdrachten van respectievelijk 45,1 en 44,0 seconden, waarbij de vertraging te verklaren is door API-responses die volledig door het model moeten worden verwerkt.

De SalesforceAgent vertoont eveneens uitschieters tot 76,5 seconden. Opmerkelijk is dat beide piekwaarden samenvallen met queries die uiteindelijk lege resultaten retourneren (score 1). De vertraging wordt hier dus niet veroorzaakt door grote data-overdracht, maar door herhaalde mislukte API-aanroepen. Dit maakt deze gevallen dubbel kostbaar: lang en inhoudelijk leeg.

De SmartSalesAgent is het meest voorspelbaar in gedrag, met een standaardafwijking van 9,2 seconden en een maximum van 43,8 seconden. De langste responstijd betreft een query naar alle SmartSales-locaties, waarbij een grote datapagina wordt teruggestuurd.

De OrchestratorAgent toont een duidelijk verschil tussen enkelvoudige en multi-agent queries: queries waarbij slechts één subagent werd aangesproken, werden gemiddeld in 15,5 seconden beantwoord, terwijl multi-agent

queries gemiddeld 29,3 seconden vergden. Elke bijkomende subagentaanroep voegt inferentietijd en netwerklatenties toe.

### 7.4.2 Evaluatie van het tokenverbruik

Het tokenverbruik verschilt substantieel tussen de agenten en weerspiegelt de architecturale complexiteit van elke subagent. De GraphAgent verbruikt gemiddeld 7.405 tokens per query (6.430 input, 975 output), de SmartSalesAgent 6.375 tokens (5.170 input, 1.205 output). De SalesforceAgent verbruikt aanzienlijk minder met gemiddeld 3.351 tokens (2.946 input, 405 output), en de OrchestratorAgent 2.703 tokens (2.266 input, 437 output).

Het hogere inputverbruik van de Graph- en SmartSalesagent is te verklaren door omvangrijkere toolbeschrijvingen en rijkere API-responses die als context worden meegestuurd. De hogere outputtokens van de SmartSalesAgent (gemiddeld 1.205) ten opzichte van de SalesforceAgent (405) weerspiegelen de inhoudelijk uitgebreidere antwoorden: SmartSales-responses bevatten doorgaans meer opgesomde data, terwijl Salesforce-antwoorden beknopt of leeg zijn. De lage outputtokens bij Salesforce zijn aldus een symptoom van het kwaliteitsprobleem.

De maximale tokenwaarden zijn het meest illustratief voor de uitschieters. De GraphAgent bereikte een piek van 49.581 tokens voor de e-mailbodyquery, grofweg zeven keer het gemiddelde van die agent. Dit gegeven illustreert hoe ongecontroleerde toolaanroepketens aanzienlijke kostpieken kunnen veroorzaken, zelfs in een enkel geval. De SmartSalesAgent bleef begrensder met een maximum van 11.594 tokens.

Bij de OrchestratorAgent stijgt het tokenverbruik met het aantal betrokken agenten: multi-agent queries vergen gemiddeld 3.214 tokens tegenover 1.809 voor enkelvoudige routingtests. Tegelijk scoren multi-agent queries gemiddeld 3,93 tegenover 3,50. De hogere investering in tokens en tijd gaat dus gepaard met een merkbaar betere antwoordkwaliteit, wat intuïtief verklaarbaar is: vragen die meerdere databronnen vereisen, worden ook rijker beantwoord wanneer de betrokken subagenten voldoende data teruggeven.

In termen van efficiëntie is de SmartSalesAgent het sterkst: een hoge gemiddelde score (4,18) bij een gemiddeld tokenverbruik dat vergelijkbaar is met de GraphAgent, maar met lagere outputtokens. De GraphAgent is minder efficiënt door de aanwezigheid van extreme uitschieters die het gemiddelde vertekenen. De SalesforceAgent is het minst efficiënt: het beperkte tokenverbruik levert geen kwaliteitsvoordeel op, en het kernprobleem, lege API-responses door onvolledige SOQL-queries, kan niet worden opgelost door meer tokens in te zetten.

## 7.5 Beperkingen van de evaluatie

De evaluatieresultaten bieden een duidelijk en kwantitatief onderbouwd beeld van de werking van het prototype binnen de gekozen testset, maar moeten in hun juiste context worden geïnterpreteerd.

Een eerste beperking betreft de SalesforceAgent. De lage gemiddelde score (2,59) en het hoge aantal lege API-

responses zijn in de eerste plaats te herleiden tot tekortkomingen in de toolconfiguratie: onvolledige veldselecties in SOQL-queries en gebrek aan ondersteuning voor relationele queries. Dit zijn correcteerbare implementatiefouten die de prestaties van de agentarchitectuur als zodanig negatief vertekenen. De Salesforce-gerelateerde Orchestrator-scores zullen verbeteren zodra de Salesforce-tools worden bijgewerkt.

Een tweede beperking is de afhankelijkheid van een LLM als evaluator. Hoewel deze aanpak schaalbaar en genuanceerd is, introduceert het subjectiviteit. De rationales in de dataset zijn in de meeste gevallen inhoudelijk verdedigbaar, maar incidentele inconsistenties zijn niet uit te sluiten. Volledig objectieve correctheidsbeoordelingen zijn bij vrije-tekstoutput inherent moeilijk te garanderen.

Tot slot zijn de gebruikte databronnen dynamisch. De data in Microsoft 365, Salesforce en SmartSales kan veranderen over de tijd, waardoor dezelfde benchmarkvragen op een later moment andere scores kunnen opleveren zonder dat het systeemgedrag is veranderd. Dit geldt in het bijzonder voor e-mailqueries, die afhankelijk zijn van de actuele mailboxinhoud, en voor Salesforce-queries die filteren op status of datum.

# 8

# Werkpunten voor een volgende versie

Deze eerste versie van het prototype toont aan dat de voorgestelde multi-agentarchitectuur technisch haalbaar is voor enterprise search over meerdere bedrijfsdatabronnen heen. Tegelijk blijft het huidige systeem op verschillende punten beperkt. In een volgende iteratie zal de nadruk liggen op het versterken van de robuustheid, het uitbreiden van de functionele mogelijkheden en het verder onderbouwen van de gemaakte architecturale keuzes.

## 8.1 Algemene uitbreidingen

Een eerste belangrijk aandachtspunt is het beheer van context en sessies over meerdere interacties heen. In de huidige versie is die ondersteuning beperkt en agent-specifiek, terwijl een volgende iteratie nood heeft aan gedeeld sessiebeheer op systeemniveau. Daardoor zouden eerdere vragen, tussenresultaten en relevante entiteiten over meerdere interacties heen behouden en hergebruikt kunnen worden.

Daarnaast is ook een robuustere foutafhandeling noodzakelijk. Een uitgebreidere versie moet beter kunnen omgaan met time-outs, mislukte toolaanroepen en gedeeltelijke resultaten wanneer één databron tijdelijk niet beschikbaar is.

## 8.2 Uitbreidingen voor de Microsoft Graph-Agent

Voor de Microsoft Graph-Agent ligt een eerste prioriteit in het aanpakken van ongecontroleerd tokenverbruik. De evaluatie toonde een extreme uitschieter waarbij het ophalen van een volledige e-mailbody resulteerde in 49.581 tokens en een verwerkingstijd van meer dan 100 seconden. Een volgende versie moet guardrails voorzien zodat dergelijke uitschieters worden voorkomen.

Een tweede verbeterpunt betreft de interpretatie van datums in kalenderqueries. In de huidige versie worden termen als *this week* of *next week* soms onjuist omgezet, wat leidt tot antwoorden buiten de gevraagde periode.

Verder is ook verbetering van het ophalen en inlezen van documenten een belangrijk uitbreidingspunt. Daarbij

kan verder onderzocht worden of Azure AI Search of lichtere alternatieven voor vectoropslag de meest geschikte aanpak vormen. Ook het contextbeheer kan worden uitgebreid: in de huidige versie is dit voornamelijk beperkt tot bestandsgerelateerde interacties, terwijl een volgende iteratie gelijkaardige ondersteuning kan voorzien voor e-mails, contacten en agenda-items.

## 8.3 Uitbreidingen voor de Salesforce-Agent

De evaluatie toont dat de lage scores van de SalesforceAgent in de eerste plaats het gevolg zijn van concrete implementatieproblemen. De veldselecties in de huidige SOQL-queries zijn op sommige plaatsen te beperkt. Daardoor geeft de agent geregeld onvolledige of lege antwoorden, zelfs wanneer de gevraagde informatie eigenlijk beschikbaar is. Deze technische tekortkomingen vormen daarom een eerste prioriteit voor een volgende versie.

Daarnaast kan de agent ook functioneel verder worden uitgebreid. Zo kunnen de filtermogelijkheden verbeterd worden en kunnen bijkomende Salesforce-objecten ondersteund worden. Ook sessiebeheer biedt nog ruimte voor verbetering. Het kan bijvoorbeeld nuttig zijn om eerder gevonden accounts, contacten of opportunities tijdelijk bij te houden, zodat die in latere interacties opnieuw gebruikt kunnen worden.

## 8.4 Uitbreidingen voor de SmartSales-Agent

De evaluatie toont dat de SmartSalesAgent consistent goed presteert op enkelvoudige queries, maar faalt op multi-step queries waarbij het resultaat van een eerste toolaanroep als invoer moet dienen voor een tweede. In elk geval slaagde de agent er niet in om de uitvoer van de eerste stap correct door te geven. Dit wijst op een beperking in de chaining-logica die in een volgende versie gericht moet worden aangepakt, zonder dat de toegang tot de onderliggende API hiervoor hoeft te worden aangepast.

## 8.5 Uitbreidingen voor de orchestrator

De orchestrator vormt het centrale coördinatiepunt van het systeem en is daarom een belangrijk onderdeel voor verdere uitbreiding. Een eerste aandachtspunt is het verbeteren van de query routing. In de huidige versie gebeurt deze volledig via het taalmodel. In een volgende iteratie kan onderzocht worden of een hybride aanpak, waarbij eenvoudige gevallen via expliciete regels worden afgehandeld en complexere gevallen via de LLM, betere resultaten oplevert.

Daarnaast vormt ook de logica voor herhaalde subagentaanroepen een verbeterpunt. Iteratieve aanroepen zijn op zich niet problematisch en kunnen nuttig zijn wanneer een vervolgstap voortbouwt op eerder verkregen informatie. De evaluatie toonde echter ook een geval van over-routing waarbij de orchestrator de SmartSalesAgent

driemaal aanriep voor dezelfde deelvraag, zonder dat daarbij een zinvolle verfijning van de informatiebehoefte optrad. Een volgende versie moet dergelijke mislukte herhalingen kunnen detecteren en tijdig afbreken, in plaats van ze verder te blijven herhalen.

Tot slot is ook sterkere ondersteuning nodig voor queries die meerdere databronnen combineren. Een volgende iteratie kan explicieter tussenresultaten bijhouden, entiteiten over databronnen heen koppelen en de samenwerking tussen gespecialiseerde agents gerichter aansturen.

## 8.6 Uitgebreidere evaluatie

Naast inhoudelijke uitbreidingen is ook een uitgebreidere evaluatie noodzakelijk. Een eerste mogelijke uitbreiding is een vergelijking tussen verschillende taalmodellen, bijvoorbeeld tussen `gpt-4o-mini` en een krachtiger alternatief, om de impact op correctheid, performantie en kost te analyseren.

Daarnaast is voor de Microsoft Graph-Agent een vergelijking tussen klassieke retrieval, semantische retrieval en Retrieval-Augmented Generation (RAG) relevant, zodat duidelijker kan worden nagegaan welke aanpak het meest geschikt is voor documentgerichte zoekvragen.

# References


[1] T. B. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[2] OpenAI, "A Practical Guide to Building Agents," OpenAI, Tech. Rep., 2025. [Online]. Available: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[3] Google Cloud, "What are AI agents?" https://cloud.google.com/discover/what-are-ai-agents, 2024, accessed: 2025.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[5] X. Qiu, H. Wang, X. Tan, C. Qu, Y. Xiong, Y. Cheng, Y. Xu, W. Chu, and Y. Qi, "Towards Collaborative Intelligence: Propagating Intentions and Reasoning for Multi-Agent Coordination with Large Language Models," *arXiv preprint arXiv:2407.12532*, 2024. [Online]. Available: https://arxiv.org/abs/2407.12532

[6] Microsoft, "AI agent design patterns," https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns, 2024, microsoft Azure Architecture Center. Accessed: 2025.

[7] LangChain, "LangGraph Documentation," https://langchain-ai.github.io/langgraph/, 2024, accessed: 2025.

[8] ——, "How and when to build multi-agent systems," https://blog.langchain.com/how-and-when-to-build-multi-agent-systems, 2024, accessed: 2025.

[9] Microsoft, "What is Semantic Kernel?" https://learn.microsoft.com/en-us/semantic-kernel/overview/, 2024, microsoft Learn. Accessed: 2025.

[10] ——, "AutoGen: Multi-Agent Conversation Framework," https://microsoft.github.io/autogen/stable/, 2024, accessed: 2025.

[11] VentureBeat, "Microsoft retires AutoGen and debuts Agent Framework to unify and govern," https://venturebeat.com/ai/microsoft-retires-autogen-and-debuts-agent-framework-to-unify-and-govern, 2024, accessed: 2025.

[12] Anthropic, "Model Context Protocol Specification," https://github.com/modelcontextprotocol/modelcontextprotocol, 2024, accessed: 2025.

[13] Hugging Face, "MCP Course – Unit 1: Architectural Components," https://huggingface.co/learn/mcp-course/unit1/architectural-components, 2024, accessed: 2025.

# 8 References

[14] Microsoft, "Introduction to AI agents and the Model Context Protocol (MCP)," https://learn.microsoft.com/en-us/azure/developer/ai/intro-agents-mcp, 2024, microsoft Azure Developer Documentation. Accessed: 2025.

[15] Red Hat, "Model Context Protocol (MCP): Understanding security risks and controls," https://www.redhat.com/en/blog/model-context-protocol-mcp-understanding-security-risks-and-controls, 2024, accessed: 2025.

[16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," *arXiv preprint arXiv:2306.05685*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

# Appendices

# Bijlage A – Volledige lijst van testqueries

Onderstaande lijst bevat alle 88 testqueries die gebruikt werden voor de evaluatie van het prototype, gegroepeerd per agent en subcategorie.

## GraphAgent (22 queries)

### Identity

- Who am I in Microsoft 365?

### E-mail

- Show me my 5 most recent emails.
- Search for emails that have the word 'meeting' in the subject.
- Have I received any emails in the last 7 days? List sender and subject.
- What does my most recent email say? Give me the full body.
- Show me emails I received in the last 14 days.
- Find emails with 'invoice' in the subject.
- Find the most recent email with 'meeting' in the subject and read its full body.

### People

- Find the email address of Dorian.
- Look up the email address of Marie in the organization.

### Kalender

- What are my upcoming calendar events this week?
- Search my calendar for any events or meetings in the next 14 days.
- How many calendar events do I have scheduled this week? List them all.
- Search my calendar for events scheduled in the next 30 days.

### Contacten

- Show me my Microsoft 365 contacts.
- List my Microsoft 365 contacts and include their phone numbers.

### Bestanden

- Find any Excel or PDF files in my OneDrive.
- Search OneDrive for a file called 'report' and read its content.

- Find all Word documents in my OneDrive.
- Find a document called ‘agenda’ in OneDrive and show its content.
- Search OneDrive for Excel files and read the content of the first two found.
- Find any files related to ‘budget’ in OneDrive and read all of them.

## SalesforceAgent (22 queries)

### Accounts

- List 5 Salesforce accounts.
- Find Salesforce accounts in Belgium, including their billing address.
- Find Salesforce accounts that have ‘Group’ in their name.
- Show me Salesforce accounts with their phone numbers.
- Find Salesforce accounts in France and list the contacts linked to those accounts.

### Contacts

- Show me 5 Salesforce contacts with their email addresses.
- Find Salesforce contacts in the Sales department.
- Find Salesforce contacts with ‘Manager’ in their title.
- Show me the most recently created Salesforce contacts.

### Leads

- Show me 5 leads in Salesforce.
- Find Salesforce leads from the Technology or Software industry.
- Find Salesforce leads that have been converted.
- Show me Salesforce leads with the status ‘Open - Not Contacted’.

### Opportunities

- List 5 open opportunities in Salesforce.
- Show me Salesforce opportunities with an amount greater than 10,000. Include probability.
- Show me Salesforce opportunities in the ‘Proposal/Price Quote’ stage.
- List Salesforce opportunities sorted by amount, highest first.
- Show me the 3 largest open Salesforce opportunities and the billing country of each related account.

### Cases

- List open support cases in Salesforce.
- Show me 5 closed Salesforce cases including their close date and description.

- Find high-priority open cases in Salesforce.
- Show me Salesforce cases with their related account names.

## SmartSalesAgent (22 queries)

### Locaties

- List all SmartSales locations.
- Find SmartSales locations in Brussels.
- List SmartSales locations in Belgium sorted by name, using full projection.
- List all SmartSales locations, then retrieve the full details of the first result.
- What fields are available to display in the SmartSales location list?
- What fields can I use to filter SmartSales locations?
- What fields can I use to sort SmartSales locations?

### Catalogus

- List SmartSales catalog items.
- List SmartSales catalog items with simple projection, sorted by name.
- List SmartSales catalog items, then retrieve the full details of the first one.
- What fields can be displayed for SmartSales catalog items?
- What fields can I filter SmartSales catalog items by?
- What fields can I sort SmartSales catalog items by?

### Orders

- List recent SmartSales orders.
- List SmartSales orders with full projection.
- List SmartSales orders, then retrieve the full details of the first order.
- What is the SmartSales order configuration?
- List all SmartSales order approbation statuses.
- List SmartSales approbation statuses, then get the full details of the first one.
- What fields can be displayed for SmartSales orders?
- What fields can I filter SmartSales orders by?
- What fields can I sort SmartSales orders by?

## OrchestratorAgent (22 queries)

### Routing queries

- Who am I in Microsoft 365?
- List 5 Salesforce accounts.
- List SmartSales locations.
- What calendar events do I have next week?
- Search my emails for anything about budget.
- Find all open Salesforce opportunities with an amount above 10,000.
- List SmartSales catalog items.
- Show me recent SmartSales orders.

**Cross-system queries**

- Find contacts named 'John' in both Microsoft 365 and Salesforce.
- Show me my calendar events for the next 7 days and check whether any organizers appear as contacts in Salesforce.
- List SmartSales locations in Brussels and check if there are any matching Salesforce accounts in the same city.
- What are my 3 most recent emails? Are any of those senders listed as contacts in Salesforce?
- Show me open Salesforce opportunities and SmartSales locations in Belgium. Give me a summary of both.
- Find my 5 most recent emails and check if any of the senders are Salesforce leads.
- List my Microsoft 365 contacts and check if any of them are also SmartSales locations.
- Find Salesforce accounts in Brussels and compare with SmartSales locations in the same city.
- Show me my calendar events for the next 7 days and check if any attendees are Salesforce contacts.
- Who am I in Microsoft 365, what Salesforce accounts exist, and what SmartSales locations are in Belgium?
- List recent SmartSales orders and check if the customer names match any Salesforce accounts.
- Search my OneDrive for files about contracts and check if the mentioned company names are Salesforce accounts.
- Find emails I received this week and check if any senders are linked to a SmartSales location.
- Give me an overview of my upcoming meetings from Microsoft 365, related Salesforce opportunities, and SmartSales locations for the companies involved.